J Primer

作者: Eric Iverson　　翻译: Taigua

目录

1 从这里开始 6

2 为什么学J? 7

3 本书的目标 9

4 需要的背景知识 10

5 如何使用本书 11

6 开始 12

7 实验 13

8 标准配置 15

9 术语 16

10 字母表 17

11 单词 19

12 句子 20

13 动词 21

14 名词 22

15 数字 23

16 负数 25
17 原语 27
18 名字 28
19 注释 30
20 错误 31
21 双重性 32
22 双子 33
23 单子 34
24 NuVoc - 新词汇表 35
25 检查点一 38
26 数值常量 39
27 字符串 40
28 单词构成 42
29 空格 45
30 优先级 48
31 括号 49
32 求值顺序 50
33 动词定义 53
34 单子/双子定义 56
35 脚本 59
36 局部名字 64

37 全局名字 66

38 调试全局名字 70

39 =. 与=: 何时类似 72

40 =. 与=: 何时不同 73

41 区域 74

42 z区域 78

43 脚本加载 80

44 检查点二 82

45 调试——逐步执行 83

46 调试——处理错误 86

47 比较原语 89

48 控制结构 91

49 检查点三 95

50 列表相加的基础方式 97

51 列表相加的J方式 102

52 更多原语 106

53 绘图 109

54 绘图区域 112

55 打印精度 114

56 不精确数值 117

57 容差 119

58 检查点四 122

59 原子 123

60 列表 124

61 表格 125

62 阵列 127

63 轴 129

64 形 130

65 秩 131

66 空阵列 132

67 单原子阵列 135

68 动词参数 137

69 框架和单元格 139

70 项 141

71 k元格 143

72 动词的秩 144

73 秩协商 146

74 秩连词 ” 150

75 结果的形 154

76 检查点五 156

77 副词 157

78 插入副词 158

79 表格化副词 163

80 连词 169

81 副词和连词的执行顺序 170

82 装盒 - 单子 < 172

83 链接 - 双子 ; 175

84 开盒 - 单子 > 178

85 取自 - 二元 { 182

86 取自（盒装索引） 184

87 取自（分散索引） 189

88 修订 }（修改选中的） 191

89 取自 { 之外的选择方式 195

90 分割 ;. 199

91 每个 each 203

92 挂钩 206

93 分叉 208

94 隐式定义 210

95 显式-隐式转换器 216

96 检查点六 220

97 外部 !: 221

98 文件 223

第 1 章

从这里开始

J是一种通用的高级编程语言。如果您是J的新手，并且想成为一名J程序员，那么这是一个很好的起点。 即使您有相当丰富的编程经验，J也有很多独特之处，在深入研究之前，至少浏览一下这些内容是值得的。

第 2 章

为什么学J?

J是一种非常丰富的语言。你可能学习和使用它很多年，但仍然认为自己是个初学者。这与Basic或Java等更简单的语言形成鲜明对比， 在这些语言中，数月的努力学习和使用将使您成为专家。成为一名专家J程序员所需要的努力与成为一名专家c++程序员所需要的努力更接近。

好消息是，J的本质是如此简单和一致，你可以很快地学会解决真正有趣的问题。

学习足够的Basic或Java来解决琐碎的问题更容易，但学习足够的J来解决更有趣和更具挑战性的问题更容易。 一旦你掌握了这种水平的技能，你就不会走到路的尽头，而是可以继续提高，使自己成为一个更好、更强大的程序员。

J尤其擅长对阵列数据进行数学、统计和逻辑方面的分析。它是一个强大的工具，可以为老问题构建新的更好的解决方案， 甚至可以更好地找到尚未完全理解的问题的解决方案。

作为一种通用编程语言，J系统还提供：

- 两个集成的开发环境。JHS和JQt
- 标准库、实用程序和包
- 为应用程序构建事件驱动图形用户界面（JQt）
- 几种与其他编程语言和应用程序接口的方法
- 快速应用程序原型和开发
- 免版税发布应用程序的运行时版本

如果你对具有挑战性的数据处理问题的编程解决方案感兴趣，那么在学习J上投入的时间将是值得的。

第 3 章

本书的目标

NuVoc是J语言的权威和明确的规范。J字典也可以用来学习J，但正因为它简洁、完整且严谨地涵盖了这门语言，并且更侧重于复杂之处而非寻常之处，它确实把我们中的一些人吓跑了。

本书为初学者提供了一个更亲切、更温和的起点。它带着你沿着一条路径，以轻松的步骤逐步前进，直到你能够用J语言编写应用程序。 在这个过程中，你会通过简化和具体的情境来接触J的所有关键概念。这本手册的目的是让你快速上手，达到能够使用NuVoc和J字典的程度， 并且用起来会让你纳闷：当初为什么还要为这些简单的东西费心呢？

你应该能够相当快地完成入门，最后将成为入门级的J程序员。这样，你将比最有经验的Basic或Java程序员拥有更强大的编程能力。

第 4 章

需要的背景知识

本书假设你熟悉另一种编程语言，如Basic、Java或C。然而，这不是先决条件，即使J是你的第一门计算机语言， 应该也不会有什么特别的问题（实际上，更应祝贺你！）。

大多数事情都可以在J中完成，就像在其他语言中完成一样，并且通常一个主题的引入方式和在其他语言中引入它一样， 这使得在J中更容易理解它是如何工作的。在某些情况下，有一种更好的J方法来解决问题，也会介绍这种方法。

第 5 章

如何使用本书

本书是一系列小的，易消化的部分，适合从头到尾按顺序阅读。一个章节通常需要你读过大部分甚至所有之前的章节。 跳来跳去毫无意义，而且可能令人沮丧。

本书是自包含的，可以在不访问J系统的情况下读取。特别是，与J系统交互的示例显示了输入的内容以及系统如何响应。 然而，它的目的是在访问系统和尽可能多地使用系统的情况下读取。强烈建议你最后输入所有示例，并尽可能多地把玩它们的变体。

有时章节使用的术语和概念直到后面才定义。这就要求你在第一次阅读时有一个初步的理解并继续，在第二次阅读时它们将变得更加具体。

本书最好读三遍：

- 浏览整本书。可以尝试一些例子，但最好持续前进，了解全局。
- 第二次仔细阅读并尝试所有的例子。
- 第三次尝试自己的例子来澄清你的理解，并提升你实际使用系统机制的舒适度（而不仅仅是阅读它和遵循说明）。

第 6 章

开始

本章假设你在JQt IDE（交互式开发环境）中使用最新版本的J。

Term窗口是一个执行窗口。你在Term窗口中键入J语句，当按Enter键时，J将执行它们并显示结果。

在Term窗口中键入以下行并按Enter键。

```
   2 + 3
```

语句被执行并显示结果。输入以下行并按Enter键。

```
   5 - 3
```

你的会话应该看起来像这样：

```
   2 + 3
5
   5 - 3
2
```

你输入的行缩进三个空格，J的答案从行首开始。

第 7 章

实验

我们鼓励你去尝试。尝试用不同的数字输入类似的行。很明显，+是加法，-是减法。 输入一些用*表示乘，用%表示除的行。通过使用%，你将很快看到结果可以是2.5这样的数字，并且它们也可以用作参数。

在你有更多的经验之前，你有时可能会对你所观察到的东西感到惊讶甚至不安。一步一步来。尝试一些你确信你已经知道答案的例子， 并做实验来证实你的理解。如果一个结果让你太困惑，早期阶段不要花时间在它上面。

```
   3 - 5

_2
```

_2，而不是-2，可能会让你困惑。别担心，一会儿就会解释的。

本书中的大多数示例会显示你应输入的内容（缩进三个空格），同时也会显示系统返回的结果。这意味着你可以轻松地通读本入门手册， 无需实际操作系统即可查看结果。然而，真正学会的唯一途径，最终还是靠你亲自尝试这些示例，并用自己的想法进行实验。 示例采用等宽字体显示，与它们在系统Term窗口中的呈现方式基本一致。部分示例会使用较大的字体，以便你更轻松地阅读并在系统中 输入——这通常出现在你可能因对某些词汇不熟悉而输错内容，或者输入错误可能导致令人困惑的结果时。

在做实验的时候，你经常想对你已经输入过的句子做一些细微的改变。有几个捷径可以使这更容易。在Term窗口中， 你可以将光标移动到窗口中

的任意行，然后按Enter键，将该行作为窗口底部的新行，以便进行编辑。通过按住Ctrl+Shift并按向上箭头键，可以调出以前的输入行进行编辑，持续按向上箭头直到看到想要处理的行。

大多数小节中的示例都是独立的，但小节的后面部分可能依赖于前面部分中采取的步骤。 有些小节依赖于前面几节中采取的步骤，但这应该是相当明显的。

第 8 章

标准配置

本书中的示例假设系统在JQt IDE中启动时运行标准配置文件。这将配置你的系统并提供一些标准实用程序。

在Term窗口中输入CR（大写字母C后跟大写字母R），以快速检查配置文件是否已运行。

```
   CR
```

如果运行结果是空行，那么标准配置文件已经运行，你可以跳过本节的其余部分。

相反，如果你看到：

```
   CR

|value error
```

那么你必须更改系统，以便运行标准配置文件。一个正常的J安装总是被配置为运行标准配置。 参考安装J。

第 9 章

术语

所有的编程语言都与英语有共同之处。在类比含义接近的情况下，J编程语言倾向于使用英语术语，而不是数学和其他编程语言中使用的术语。

像在其他语言中一样，你可以说“代码行”，但在J语言中，倾向于说“句子”。类似地，你可以把+称为“函数”，但在J中通常称为“动词”。

J使用的英语术语有：字母表、单词、句子、短语、动词、名词、副词和连词。

采用这种方法有几个原因。它解决的一个问题是：传统术语在数学和许多编程语言中有大量相关但又微妙不同的用法。 例如：函数、子函数、运算符、程序、例程和子例程在不同的编程语言中都以略有不同的方式使用。J没有继承这种混乱，而是采用自己的术语，并在其上下文中精确地定义它们。

使用英语术语可以让你很好地了解术语在J中的一般含义。此外，使用自然语言术语可以鼓励和促进对问题进行英语陈述， 并更直接地编写相应的J句子。

我们鼓励使用J术语，但肯定不是强制性的，使用术语“函数”而不是“动词”是完全可以接受的。

第 10 章

字母表

J编程语言的字母表是ASCII字母表，由以下部分组成：

```
26 lowercase letters (a to z)

26 uppercase letters (A to Z)

0 1 2 3 4 5 6 7 8 9

 = < > _

 + * - %

 ^ $ ~ |

. : , ;

# ! / \

[ ] { }

" ` @ & ?

( )
```

'

有几个字符有时会引起混淆。-（减号）字符不同于_（下划线）字符，并且有三种不同的引号字符：

' quote

" double-quote

` back-quote

如果你尝试一个例子，输入"（双引号）误输入为''（两个单引号），你会感到失望，因为你的结果与书中的不一样。

句号.通常称为点。

第 11 章

单词

单词是字母表中有意义的一组字符。

```
2.5 + 5
```

这个句子有三个单词：数字2.5，+和数字5。

用字符组成句子中的单词的规则很简单，目前，常识足以识别句子中的单词。有一些复杂情况在后面的章节中讨论。

第 12 章

句子

句子是由一组单词组成的完整指令。
下面的2加5指令就是一个J语言的句子。

```
2 + 5
```

与英语句子不同，J语言的句子不以句号或其他标点符号结尾。
相反，J编程语言的句子通常是一个完整的行。

第 13 章

动词

在下面的句子中+是动词（表示动作的单词）。

```
2 + 5
```

第 14 章

名词

在下面的句子中，数字2和5都是名词。

```
2 + 5
```

第 15 章

数字

以下这些是数字：0 12 02.5 12.75 0.5 7e6

输入这些数字或其他数字，用动词+ - * %组成简单句子。

```
  2 * 12.75
```

```
25.5
```

一个重要的规则是数字不能以点（.）开头。

```
  0.5
```

```
0.5
```

```
   0.5 + 3
```

```
3.5
```

```
   .5
```

```
+-+-+
```

```
|.|5|
```

```
+-+-+
```

```
   .5 + 3

|syntax error

|         .5+3
```

显然，.5和0.5不一样。你现在不需要知道.5是什么，但重要的是明白数字不是以点开头的。

_（下划线）是无穷大，是一个数字。

第 16 章

负数

负数用_（下划线）表示，而不是-（减号）。

```
   5 + _3
```

```
2
```

```
   _7
```

```
_7
```

```
   _5e_3
```

```
_0.005
```

```
   5 - 9
```

```
_4
```

去掉-和9之间的空白并不会改变含义。

```
   5 -9
```

```
_4
```

-总是动词。这简化了识别句子的规则，因为没有把-直接用在数字左边这种特殊情况。 代价就是，既然-总是一个动词，则需要另一个字符_（下划线）来拼写负数。

_用于拼写数字，放在有0 1 2 3 4 5 6 7 8 9，．以及表示指数符号的e组成的数字前；表示一个负数。

记住：数字前面的•_•是数字的一部分，表示它是负数，而-总是一个动词，不是数字的一部分。

__（两个下划线）表示负无穷，是一个数字。

第 17 章

原语

原语是由系统定义的一个单词。例如，+是一个原语。原语的含义是固定的，不能改变。

原语可以由一个图形字符（如+）拼写的，或者用图形字符加上一个表示变形的字符（点号或冒号）拼写，如+.或者+:。

原语也可以由一个或多个字母后跟一个点号或冒号拼写。例如，i.是一个原语，根据它的使用方式的不同，被称为**生成索引**或**求索引**。

第 18 章

名字

原语是由系统定义的单词，而名称是由你定义的单词。原语=.定义一个名字。

```
   5 + v
```

```
28
```

这个句子可以读作v是23。单词=.叫做**系动词**（另一个很好的英语术语）。名字v被定义为数字23，可以用在其他句子中。

与原语不同，名字可以重新定义。

```
   v =. 45

   5 + v
```

```
50
```

当句子以系动词开始时，系统不会显示句子的结果。只包含名字的句子将显示名字所定义内容的显示形式。

```
   abc =. 123

   abc
```

```
123
```

你可以给任何东西起名字。例如，你可以给动词+起一个名字。

```
   plus =. +

   23 plus 45
```

```
68
```

abc =. def的一般读作：abc被定义为def。然而，借用其他计算机语言，也常说：abc被赋值为def或将值def赋给abc。

第 19 章

注释

原语NB.开始一个注释，该注释一直到行尾结束。

```
   2 + 23   NB. this is a comment and is not executed
```

```
25
```

第 20 章

错误

句子中的错误将会被报告并停止执行。

```
   123 foo 234

|value error: foo

|   123     foo 234
```

一条竖线标记了错误报告。如果可以的话，句子会用额外的空格来标记检测到错误的地方。 在J9.4版本中，错误消息得到了改进，使调试更加容易。

第 21 章

双重性

每个动词都有两种定义：当它同时有左参数和右参数时使用双子（dyad），当它只有右参数时使用单子（monad）。 这两个定义通常是相关的，比如-的双子定义表示减法，而单子定义表示求负。

```
   5 - 3
2
   - 7
_7
```

双子也被称为动词的二元模式，而单子被称为动词的一元模式。 “双重性”一词用于化学意义上的两种价态，表明一个动词既可以与一个参数反应，也可以与两个参数反应。

记住：绝大多数动词都有单子和双子两种定义。

第 22 章

双子

如果一个动词同时有左参数和右参数，则使用动词的二元模式。

双子%（除以）被定义为左参数除以右参数。

```
5 % 2
```

```
2.5
```

第 23 章

单子

当动词只有一个右参数时，使用动词的一元模式。

单子%（倒数）被定义为1除以右边的参数。

```
    % 2
```

```
0.5
```

单子%和双子%之间的关系很常见，其中单子是带有固定左参数的双子，你会在许多其他原语动词中看到这种关系。

第 24 章

NuVoc - 新词汇表

NuVoc（菜单Help|Vocabulary）是最新的J参考， 是你关于J的最终、权威信息来源。你越早熟悉使用它越好，一个好的开始方法是查找到目前为止介绍的原语。

在JQt IDE中，按F1显示NuVoc，并单击原语跳转到条目。如果你按下F1同时按住Ctrl (Mac的Cmd)， 你会得到上下文敏感的帮助。如果光标在+处，按Ctrl+F1将跳转到原语+的条目。

在NuVoc中，查看包含+的第6行。这一行的第一项包含：+ 0 Conjugate . Plus 0 0。 单词+是动词。它的单子称为求共轭，它的双子称为加法，就像算术中定义的那样。

单子+很有趣。在数学中，复数的共轭是实部相同，虚部相反的数。在实数上求共轭没有作用，结果是参数本身， 在复数上它改变虚部的符号。J支持复数，就像直接支持整数或实数一样。复数用j分隔实部和虚部来表示。

```
   int =. 23

   + int
23

   float =. 23.5
```

```
   + float
```

```
23.5
```

```
   imagine =. 2j3    NB. 2 real part, 3 imaginary part
```

```
   + imagine         NB. change sign of imaginary part
```

```
2j_3
```

许多原语支持复数，NuVoc必须对此进行记录，这意味着在某些描述中存在一些额外的复杂性。 如果你的应用程序中需要复数，这是非常棒的。但如果你是一个初学者，不关心复数， 那么你必须知道足够的知识，从而能够忽略掉这些，而不会分心或困惑。

让我们看一下wiki中的Plus条目。你可以通过点击Plus链接访问。这将带你到描述+的页面中间。 wiki页面的上半部分是描述Conjugate的信息。Plus的标题行显示它是一个带参数x和y的双子动词。 下面是一个指向Rank 0 0的链接，以及一个指向为什么这很重要的链接。你很快就会知道的。

接着下面是+的简要描述。然后，该页面继续进行一般性讨论和示例。

回到NuVoc，看看+列右边的+.条目，它包含：+. Real/Imaginary . GCD (or)， 其中Real/Imaginary是双子，GCD (or)是单子。我们来看一下定义。

参考页面的上半部分涵盖了+.的Real/Imaginary用法。下半部分覆盖GCD (or)。

二元模式被描述为：GCD (Or)，它有两个非正式的名称，GCD和Or，表明它可以以两种不同的方式使用。 请注意，GCD代表最大公约数（这至少应该唤起数学记忆）。进一步在定义中，你将发现，如果参数是布尔值，则+.是逻辑or函数。 GCD是将or函数的域拓展到非布尔参数的有用扩展。这种原语域的扩展在J中很常见。现在有趣的是，请注意+.有这个较大的域， 但也很容易把它限制到布尔参数上。

```
   0 +. 0   NB. 0 or 0
```

```
0
```

```
   0 +. 1
1
   1 +. 0
1
   1 +. 1
1
```

在NuVoc中再移动一列，我们可以看到词汇表条目+:包含: Double . Not-Or。该定义对单子和双子都给出了非常简单的定义。

一元模式的情况称为加倍，它的作用正如你所期望的那样。

```
   +:  3
6
```

二元模式是实参逻辑或运算的否定。

同样，NuVoc中的一些一般性讨论和示例现在可能超出了你的能力范围。但关键是要知道如何导航并获取相关信息。

?和?.行之后，有一些其他原语行，这些原语是由用点或者冒号作为后缀的名字组成的。 在原语表的下面是一个辅助页面列表，它们解释了J的一些概念。它们中的一些在你完成本入门之后值得一看， 因为它们提供了你将看到的一些概念的更详细的信息。

第 25 章

检查点一

到了这个检查点，你应该明白：

- 如何使用J字典词汇表
- 术语：单词、句子、名词、动词、矛盾词、双子、单子等

通过做以下练习来检查你的理解：

- 查找单子+: *: -: %:的定义
- 试试这些新单子

第 26 章

数值常量

你已经看到了单个数字的使用。也可以有一个数字列表。

```
num =. 5

nums =. 23 0.5 12.5 7e6 _12 7
```

你将会对数字列表做更多操作，但现在我们只想建立这样一个列表。

第 27 章

字符串

用单引号括起来的字符创建一个字符串。

```
   char =. 'Q'

   chars =. 'this is a list of characters'
```

显示字符串时不显示单引号。

```
   char
```

```
Q
```

```
   chars
```

```
this is a list of characters
```

单引号用于字符串的开始和结束。一对单引号表示引号字符本身在字符串中。

```
   'put 2 ''s to get 1 '' in the string'
```

```
put 2 's to get 1 ' in the string
```

不匹配的单引号是错误的。

```
   abc =. 'asdf

|open quote
```

字符串可以为空。

```
   abc =. ''
```

字符串也被称为字面值常量或字符常量。

第 28 章

单词构成

单子;:可以帮助你找出句子中的单词。单词构成原语以字符串作为其右参数，将其分割成单词，并返回结果，每个单词都在一个盒子中。 先不要担心这些盒子是什么，只需注意它们在视觉上的帮助。你将在后面的部分中了解盒子。

```
   ;: '2 + 3'
+-----+
|2|+|3|
+-----+
   ;: '2.5 + 3e4'
+---+-+---+
|2.5|+|3e4|
+---+-+---+
   ;: 'a =. 1 2 3'
+-+--+-----+
```

```
|a|=.|1 2 3|
+-+--+-----+
   ;: 'test + 123 NB. this is a comment'
+----+-+---+--------------------+
|test|+|123|NB. this is a comment|
+----+-+---+--------------------+
   ;: 'def =. ''testing 1 2 3'''
+---+--+--------------+
|def|=.|'testing 1 2 3'|
+---+--+--------------+
```

请注意，以下都是J单词，每个单词都在自己的盒子里：

```
2.5
3e4
=.
1 2 3
test
NB. this is a comment
'testing 1 2 3'
```

你可能会感到惊讶，诸如`1 2 3`和`'testing 1 2 3'`之类的常量都是J单

词。 这是很重要的一点，理解这一点在阅读和书写J句子时是必要的。

如果你确实对一个J句子感到困惑（这是可能发生的），你可以做的一件事就是对它应用;:, 以确保你知道构成它的单词。然后你就可以关注这些单词的意思了。

在NuVoc中查找;:。;:的非正式名称单词构成。

第 29 章

空格

不需要空格来分隔原语和其他单词。另一方面，额外的空格也不会改变意思。

```
   2+3

5

   2 + 3

5

   2     +     3

5
```

本书中的大多数示例在所有原语周围都有空格。这使得单个单词显而易见，让你专注于单词的意思，而不需要先弄清哪些是单词。

然而，大多数J程序员，随着他们变得越来越有经验，达到了可以轻松阅读句子中的单词的程度，而额外的空格变成了理解的麻烦和障碍，而不是帮助。 你将注意到，在后面的一些示例中，省略了一些不必要的空格。

在某些情况下，空间是必不可少的。

名字之间必须用空格分隔。

```
   a=.3

   plus=.+

   a plus a
6

   aplusa

|value error: aplusa
```

而.（点）和:（冒号）用作单词屈折变化，将紧接在它们前面的单词变成一个新词。当它们用作原语或原语的开头时， 必须在前面有一个空格，这样它们就不会被视为屈折。

```
   ;: 'a . b'  NB. 3 words

+-+-+-+

|a|.|b|

+-+-+-+

   ;: 'a .b'    NB. same 3 words

+-+-+-+

|a|.|b|

+-+-+-+

   ;: 'a. b'    NB. 2 words

+--+-+

|a.|b|
```

```
+--+-+
```

如上所示，.或者:如果不是用作屈折，则必须和它们之前的单词之间用空格分开。

数字，例如1e7，可以包含字母。事实上，在J中，有几个字母可以用于拼写数字。而紧跟着数字的字母将被当作数字拼写的一部分。

```
   plus =. +

   1plus 3

|ill-formed number

|       1plus 3

|       ^

   1 plus 3

4
```

必须用空格将数字与非数字组成部分的字母分开。

第 30 章

优先级

数学传统上认为乘法优先于加法。在数学课上，如果有人问你`2 + 3 * 4`等于多少，你要知道答案是`14`。

如果只有几个函数和几个级别的优先级(除法|乘法|加法和减法)，这并不太令人困惑。但事情在诸如C这样的语言中会变得笨拙， 因为有许多功能和优先级级别。

由于J中有大量的动词，很难定义优先级，更不用说在阅读或编写时试图记住它了。 此外，能够命名事物意味着你还必须弄清楚如何处理这个句子：

```
2 plus 3 times 4
```

与传统数学以及大多数其他编程语言不同，J没有动词优先级。你可能需要一点时间先停止乘法，但整体的简化是值得的。

记住：**没有**动词优先级。

第 31 章

括号

数学和大多数编程语言（包括J语言）都使用括号来控制句子的求值。如果一个句子被完全括起来， 那么大多数语言的求值顺序是相同的，并且不受动词优先级或任何其他规则的影响。

```
      2 + (3 * 4)
14

   (2 + 3) * 4
20

   10 - (4 - 3)
9

   (10 - 4) - 3
3
```

对这些答案没有任何异议。

第 32 章

求值顺序

如果省略括号，答案是什么？

```
   10 - 4 - 3
```

```
9
```

J对句子求值为：

```
   10 - (4 - 3)
```

```
9
```

大多数其他语言会将其求值为：

```
   (10 - 4) - 3
```

```
3
```

在没有括号的情况下，J从右到左隐式地产生语句。其他语言从左到右产生它们。 一个更长的句子会让这看起来更清晰。

```
   10 - 4 - 3 - 1
```

```
8
```

```
   10 - (4 - (3 - 1))   NB. J right-to-left
8
   ((10 - 4) - 3) - 1   NB. others left-to-right
2
```

现在考虑一组一元动词。

```
   - - - 4
_4
```

每个人都知道如何加括号，每个语言都是一样的。

```
   - (- (- 4))
_4
```

分组是从右向左进行的，在这种情况下，其他语言和J一样。J总是从右向左插入括号，而其他语言对于不同的情况有不同的规则。

J的求值顺序是从右到左。大多数其他语言都是从左到右对二元动词求值，从右到左对一元动词求值；而求值顺序又会被所涉及的动词的相对优先级修改。

对于名词和动词，J词典E章节的J求值规则是：

> 执行从右向左进行，但当遇到右括号时，先执行它及其匹配的左括号所包含的段，用其结果替换整个段及括号。

我们应该注意到，虽然NuVoc是J信息的权威来源，但J字典中有很多有用的信息可能没有进入wiki。

在J中，除了名词和动词之外，还有一些你还没见过的东西，它们会让这个规则变得更加复杂。正是这些额外的类在很大程度上证明了J打破传统 并采用从右到左的求值是合理的。进一步引自J词典E章节：

> 这些规则的一个重要后果是，在没有括号的表达式中，任何动词的右参数都是其右侧整个短语的结果。

这是因为没有动词优先级以及从右到左的求值的结果。

没有动词优先级、从右到左的求值以及其他类的规则使总体求值规则变得简单，减少了对括号的需要，并且使有经验的J用户更容易阅读和编写句子。

阅读下面的句子，在你的脑海中对它们求值，并理解无优先级和从右到左求值是如何解释答案的。

```
   2 * 4 + 5
18

   2 + 4 * 5
22

   2 - 4 - 5
3

   8 % 2 + 2
2
```

记住：没有动词优先级和从右到左的求值。

第 33 章

动词定义

编程的艺术不在于使用原语，而在于定义适合自己需求的动词。在定义自己的动词时，你是通过扩展语言来构建解决特定问题集的应用程序。

我们假设一个华氏温度和摄氏温度转换的问题。你需要定义一个动词来完成这个任务。

下面是动词摄氏度的定义，它将把它的参数从华氏温度转换成摄氏温度。

```
   centigrade =. 3 : 0

t1 =. y - 32

t2 =. t1 * 5

t3 =. t2 % 9

)
```

此时，你想要做一些有用事情，而不是处理由拼写错误引起的问题，因此请仔细抄写。 在终端窗口输入第一行：

```
   centigrade =. 3 : 0
```

按所示输入。在3和:之间必须有一个空格。3表示正在定义一个动词，0表

示定义在随后的输入行中。

当你输入上述行之后，光标位于左侧行首，并且没有像通常那样缩进三个空格。这表明系统正在等待你输入定义的其余部分。

输入示例中摄氏度定义里剩下的行。它们位于左边界处，因此看起来它们可能是由系统显示的输出，但实际上它们是定义动词所需的行输入。

最后一行仅包含)的最后一行，结束了这种特殊的定义输入模式。输入最后一行后，系统再次缩进三个空格，表示准备执行一个句子。

如果你输入的定义正确，你应该可以尝试使用你的新动词。

```
   centigrade 32

0

   centigrade _40

_40

   centigrade 212

100
```

让我们看看它的定义并理解它是如何工作的。定义的第一句中的y是动词实参的名称。当你执行带有参数的动词时， 第一行将从参数中减去32并定义t1。当第一行完成后，继续执行下一行，它将t2定义为t1乘以5的结果。 继续执行到下一行，并将t3定义为t2除以9。没有更多的行了，所以动词的执行完成了。动词的结果是最后计算的结果。

我们用3:0来定义动词。短语verb define和它是等价的，有些人觉得短语更容易阅读。事实上， 如果你在系统中键入verb或define，你将看到它们已经分别被定义为3和:0。 但是，短语隐藏了信息，而我们将使用3 : 0这种形式。

```
   centrigrade =. verb define

t1 =. y - 32

t2 =. t1 * 5
```

```
t3 =. t2 % 9

)

        verb

3

        define

:0
```

第 34 章

单子/双子定义

正如前面关于两可性的部分所讨论的那样，所有动词都有两种定义，单子和双子。 你只定义了摄氏度的单子。那么双子呢？

```
   23 centigrade 32

|domain error: centigrade

|   23     centigrade 32
```

由于你没有提供一个双子定义，它是空的，这被视为双子在其域中没有参数，你给出的任何参数都会导致域错误。

让我们看一些简单的例子来定义双子、单子和都定义的情况。

```
   monadminus =. 3 : 0

- y

)

   monadminus 5

_5

   5 monadminus 3
```

```
|domain error: monadminus

|   5     monadminus 3
```

上面的代码定义了名为monadminus的动词的单子。一元应用有效，二元应用失败。

对于这种只有一行的定义，你可以采用快捷方式。将定义放在单行上，并避免进入需要以)结尾的特殊输入模式。 以下是完成上述定义的等效方法：

```
   monadminus =. 3 : '- y'
```

到目前为止，你只定义了动词的单子。你也可以通过只定义双子来定义一个动词。使用4来定义双子，而不是将3作为:的左参数。

```
   dyadminus =. 4 : 'x - y'

   5 dyadminus 3
2

   dyadminus 5

|domain error: dyadminus

|       dyadminus 5
```

在单子的情况下，y是右参数；在双子的情况下，x是左参数，y是右参数。

如果你想同时定义一个动词的单子和双子呢？

```
   minus =. 3 : 0

- y

:
```

```
x - y
)
```

行:分隔了单子和双子的定义。

```
   3 minus 5
_2
   5 minus 3
2
   minus 5
_5
```

第 35 章

脚本

当你关闭J时，所有名字的定义都将失效。你在Term窗口中执行的操作只影响当前会话，但不是永久性的。 这在实验时很好，但是当你开始定义像摄氏度这样的动词时，会想要记录下这个定义，以便在另一个会话中使用它。

关闭J并重新启动它。

```
   centigrade
```

```
|value error: centigrade
```

因为你已经重新启动了，在上一个会话中定义的摄氏度动词和所有其他名字都丢失了。

但至少原语还在那里！

```
   2 + 5
```

```
7
```

正如你所想的那样，为了永久记录定义，要将它们保存在文件中。带有J句子和定义的文件称为脚本文件， 你可以像编辑任何其他文本文件一样编辑它们。脚本文件的后缀通常是.ijs。

记住：脚本文件是存储定义的源文件。

尽管你可以使用任何文本编辑器来处理脚本文件，但是J系统提供了一个简单的编辑器，该编辑器通过集成在J系统的方式使其更加方便使用。

当使用JQt界面时，File | New temp菜单命令创建一个新的脚本文件和一个用于编辑它的窗口。 当你点击它时，将看到你的J会话有一个Term窗口和一个新的Edit窗口来编辑.ij文件。

Edit窗口中，.ijs文件的名字显示在其标题栏。在Edit窗口中按下Enter键不会执行该行，它只是将光标移动到新行的开头。

在Edit窗口中输入你的摄氏度定义。

```
centigrade =: 3 : 0

t1 =. y - 32

t2 =. t1 * 5

t3 =. t2 % 9

)
```

在第一行**一定**要使用=:而不是=.。=:创建一个全局定义。如果使用=.，则是一个局部定义。这个重要的区别将在接下来的几页中解释。 现在，如果你希望赋值具有预期的效果，只需确保使用=:创建文件中的定义。

到目前为止，你只是将在窗口中更改编辑。该文件尚未更改，动词仍未定义。你必须通过运行脚本来执行句子。

在Edit窗口中点击Run | Load Script以将更改保存到文件中，然后执行文件中的每个句子。 这类似于在Term窗口中输入文件的内容，只是没有显示句子和结果。Term窗口中唯一显示的是系统生成的语句，它表示文件中的语句已被执行。 这句话类似于：load 'c:/j9.4-user/temp/1.ijs'，取决于你运行的J的版本。如果报告了一个错误（在Term窗口中输出，左侧有一个竖线）， 那么你的脚本中有拼写错误。在Edit窗口中更正文本并再次运行。

脚本文件中的句子已经执行完毕，现在已经定义了摄氏度动词。在Term窗口尝试使用摄氏度。

```
   centigrade 32
```

```
0
```

使用File | New temp创建的文件位于J临时目录中，并具有临时形式名称(带有.ij后缀)。 但是，现在最好在J用户目录中使用更合适的名称重新保存它。点击File | Save As...，向上一个目录级别，然后到用户目录，并将文件名设置为cf.ijs。Edit窗口标题栏中的文件名将更改为新名称。

关闭cf.ijs窗口并删除你的摄氏度定义。你可以使用功能动词erase来删除名字的定义，其参数是要删除名字的字符串。结果为1表示删除成功。

```
   erase 'centigrade'
1

   centigrade 212
|value error: centigrade
|       centigrade 212
```

使用File | Recent打开cf.ijs窗口，使用Run | Load Script加载并运行定义摄氏度的脚本。

```
   centigrade 212
100
```

让我们在cf.ijs窗口中添加一个华氏度的定义。在你的摄氏定义后输入以下内容。同样，一定要使用=:。

```
fahrenheit =: 3 : 0

t1 =. y * 9

t2 =. t1 % 5

t3 =. t2 + 32
```

)

使用Run | Load Script加载并运行cf.ijs脚本中的句子。

```
   fahrenheit 0
32
   fahrenheit 451
843.8
```

关闭J并重启。

```
   centigrade
|value error: centigrade
   fahrenheit
|value error: fahrenheit
```

使用File | Recent并选择你的cf.ijs文件。然后点击Run | Load Script，一个类似下面的行

```
   load'c:/j9.4-user/cf.ijs'
```

出现在Term窗口，表示执行了文件中的句子，而你的动词现在已经定义。

```
   centigrade 32
0
   fahrenheit 100
212
```

在Term窗口中出现的以load开头的行实际上是导致文件中的句子被执行的句子。 菜单命令只是执行这个句子的一种快捷方式。该字符串是要运行的文件的完整路径名。 手动输入完整路径名时，可以将其缩短为相对路径名。

```
   load'~/j9.4-user/cf.ijs'
```

现在检查一下你的动词是否定义好了。

使用File | Recent打开你的cf.ijs文件进行编辑。

如果脚本中有错误怎么办？让我们在脚本中添加一个故意的错误，看看会发生什么。在脚本末尾添加foo 123行，然后再次运行脚本。

```
   load'c:/j9.4-user/cf.ijs'

|value error: foo

|       foo 123

|[-13]
```

J系统报告错误并停止执行脚本中句子。错误报告末尾的数字是脚本中出现错误的行号。

从脚本中删除错误并再次运行它。

第 36 章

局部名字

动词摄氏度在其定义中使用了名字t1、t2和t3，但如果你在动词之外引用它们，它们就没有定义。

```
   centigrade 32

0

   t1

|value error: t1

   t1 =. 123

   t1

123

   centigrade 212

100

   t1

123
```

在摄氏度定义内使用t1和在定义外使用t1没有冲突。动词摄氏度并没有在它自身之外定义一个t1， 正如value error所表明的那样，给摄氏度的t1中设置一个值并不会改变它定义之外的t1的值。

摄氏度内部使用的t1是局部名字。局部名字仅存在于动词中。在摄氏度外面使用的t1是一个全局名字。 在执行动词定义是遇到的由连词=.定义的名字，是局部名字。

第 37 章

全局名字

在动词执行之外定义的名字是全局名子。

上一节中，Term窗口中定义的t1是一个全局名子，与动词摄氏度内定义的t1完全不同。

让我们来做些实验。使用File | New temp创建一个临时脚本文件，并在其中输入以下定义：

```
fooa =: 3 : 0   NB. =: is important

zzz + y

)
```

使用Run | Load script运行脚本。Term窗口显示：

```
   fooa 5

|value error: zzz

|       zzz+y
```

让我们定义全局zzz，看看会发生什么。在动词外定义它，无论我们使用=.还是=:，它都是全局的。

为了遵循最佳实践，我们将在Term窗口中使用=:：

```
   zzz =: 23  NB. define global zzz

   fooa 5
```

```
28
```

动词fooa使用全局的zzz。动词可以使用全局名字。

编辑脚本以添加“‘foob‘，然后运行脚本。

```
foob =: 3 : 0

zzz =. 7

zzz + y

)
```

在Term窗口中：

```
   foob 3
```

```
10
```

```
   zzz
```

```
23
```

动词foob使用它的局部zzz而忽略全局zzz。因此，动词可以使用局部名字而忽略同名的全局名字。

如果你想定义一个全局名字呢？全局连词=:（带:词形变化的=）定义了一个全局名字。编辑脚本以添加fooc，然后运行脚本。

```
fooc =: 3 : 0

gw =: y
```

```
lz =. y

)
```

在Term窗口中：

```
   fooc 3

3

   gw

3

   lz

|value error: lz

   gw =: 24  NB. Using =: to assign a global value, although
↪  assignment with =.  outside of a verb would also be
↪  global.

   fooc 5

5

   gw

5
```

用=:定义gw，则定义了全局名字。

一般来说，在动词中只定义局部名字而不定义全局名字是一种很好的做法。这是有时被称为函数式编程风格的重要组成部分。 定义全局名字的动词被认为具有副作用，并且更有可能导致bug，并且使阅读应用程序以理解正在发生的事情变得更加困难。 事实上，在J的最新版本中，当你定义一个同时使用同名全局名字和局部名字定义的动词并且运行它，将会触发一

个域错误。

第 38 章

调试全局名字

有时，在尝试调试或更好地理解动词时，查看其局部名字或其他中间结果的值是很有用的。 一种快速的方法是在动词定义中添加一行来进行全局定义。

打开cf.ijs文件并向摄氏度定义添加一行，将全局gt1定义为t1。

```
centigrade =: 3 : 0   NB. =:

t1 =. y - 32

gt1 =: t1             NB. temp line for debugging info

t2 =. t1 * 5

t3 =. t2 % 9

)
```

运行脚本。在Term窗口中：

```
   centigrade 124
51.1111
   t1
```

```
|value error

   gt1

92
```

在摄氏度动词完成执行后，你无法看到局部t1的值，但你可以在gt1中看到值的副本。

从脚本中删除该行，并运行脚本以重新定义不含调试行的摄氏度动词。

第 39 章

=. 与=:何时类似

你已经看到了=.和=:用于动词定义时的不同之处。

当你在Term窗口中执行句子时，你不会在动词内执行它们，此时=.和=:有相同的效果。

```
   NB. In the Term window

   a =. 123

   a
123

   a =: 234

   a
234
```

在Term窗口中连词=.和=:是相同的，你使用哪个来定义名字并不重要，因为它们都定义了全局名字。 =.更容易输入，也更容易被使用。但严格来说，在定义全局名字时，最好显式地使用=:。

第 40 章

=. 与=:何时不同

你已经看到了=.和=:在Term窗口中是相同的。同时在动词定义里，它们是不同的。 但重要的是要认识到在脚本中也存在差异。

当你运行脚本时，load句子在Term窗口中执行，动词load执行脚本中的句子。 因此，脚本中的句子在load动词中执行。这意味着用=.定义的名字被定义为动词load的局部名字。 如果想在脚本中定义一个全局名字，必须使用=:。这就是为什么在脚本cf.ijs中定义全局名字 （如摄氏度和华氏度）的行必须使用=:的原因。如果使用=.，它们将成为load的局部名字，并在load执行完后立即消失。

始终考虑定义是全局的还是局部的，并据此正确使用=:和=.。

第 41 章

区域

首先，请注意区域（locale）与局部（local）是一个非常不同的单词，尽管后者只少了一个字母。

区域是一组全局名字。可以有多个区域，因此可以有多个全局名字集。

一个区域中的全局名字与其他区域中的相同名字的区别是，通过添加用_（下划线）括起来的区域名称后缀来限定名字。 带有区域限定的名字始终是全局名字。

```
abc_def_ =: 2
```

上面的句子可以读作在def区域中的全局abc是2。

```
abc_base_ =: 4
```

上面的句子可以读作在base区域中的全局abc是4。

如果省略了区域名称，则假定它是base区域的名字。

```
abc__   NB. the same as abc_base_
```

```
4
```

如果全局名字没有使用区域名称进行限定，则它位于当前区域中。base区域是当前默认区域， 除非在不同的区域中执行动词显式地更改了当前区

域。下面的代码定义了base区域中的abc：

```
   abc =. 6

   abc_base_

6

   abc

6
```

由于base区域是当前默认区域，因此名字abc和abc_base_是相同的。

名字abc_def_显然不同于abc，但到目前为止还没有办法知道发生了什么特别的事情。 在什么意义上abc和foo在同一个（base）区域中？而abc和abc_def_在不同的区域？

一种区分方法是使用列出全局名字的工具动词names。

```
   a =. 23

   b =. 24

   a_q_ =. 25

   w_q_ =. 26

   names 0  NB. 0 lists nouns

a abc b
```

你的names结果可能不同，但它将包括你在base区域中定义的所有全局名字。 你应该看到上面定义的a和b，并注意到没有看到在q区域中定义的w。

要查看在q区域中定义的名字，你可以执行以下操作：

```
   names_q_ 0    NB. names in locale q

a w
```

名词a和w在q区域中定义。

区域将全局名字划分为不同的集合，而诸如names之类的实用程序可以操作特定区域中的全局名字。

区域的真正威力在于区域中定义的动词。当动词在一个区域中执行时，它将该区域（而不是base区域）作为当前区域。

让我们在q区域中定义一个简单的动词，看看它是如何工作的。

```
   f_q_ =. 3 : 'a =: y'
```

这个动词用右参数定义全局a。可以有许多不同的区域，每个区域都有自己的全局a。但是当f_q_执行时， 它在q区域中执行，q区域是当前的区域，它使用的全局名字来自q区域。试试下面的实验：

```
   a =. 23    NB. define a in the base locale

   a_q_ =. 24   NB. define a in locale q

   f_q_ 100 NB. execute f in locale q
100

   a
23

   a_q_
100
```

区域将全局名字划分为单独的集合。特别是，相关的名词和动词（比如在一组实用程序中），可以在它们自己的区域中定义。 它们的名字不会与base区域或其他区域中的名字冲突。当你查看应用程序时，你可以只查看特定区域中的相关全局名字。 当动词在区域中运行时，它使用来自同一区域的全局名字。

参数为0时，names动词列出名词，参数为3时列出动词，参数为6时列出区域名称。

区域名称列表很有趣。base和q大家都知道，但是j和z呢？

j和z区域中的全局名字是在J启动并运行profile.ijs脚本文件时定义的。j区域包含在构建应用程序时很有用的东西，在J在线文档中有详细讨论。

z区域确实非常有趣。

第 42 章

z区域

z区域是所有其他区域的父区域。

如果在当前区域中找不到名字，但在z区域中有它的定义，则使用该定义，就好像它在当前区域中一样。

z区域用于你希望在任何地方都可用的公共实用程序。通过z区域，它们可以在任何区域中执行，就像它们在该区域中一样， 但是只有一个副本，并且z区域中的名字不会混淆其他区域中的名字。

profile.ijs在启动J时运行，它在z区域中定义了许多标准实用程序。你已经使用了在z区域中定义的erase和names动词。 你可以通过以下方式来确认：

```
   names 3 NB. verbs in the base locale
```

...

上面的代码没有将names作为名字列出，但是你可以执行它。这是因为当在base区域中找不到它时，它在z区域中的定义就会被使用， 就好像它实际上是在base区域中定义的一样。

```
   names_z_ 3  NB. verbs in z locale
```

...

结果太长了，就不在这里列出了。动词names有一个二元定义，它接受一个左参数，该参数指示要返回的名字的第一个字母。

```
    'n' names_z_ 3

nameclass namelist names nc nl ...
```

第 43 章

脚本加载

除了与标准配置文件一起加载的实用程序之外，系统还提供了几个额外的标准实用程序脚本。 这些标准实用程序在J在线文档中有记录， 可通过J帮助菜单获得。你可以直接运行这些脚本，但是你需要记住脚本的路径，以及在哪个区域中运行它们。 标准配置文件提供了一些实用程序，使你更容易完成此类操作。scripts动词列出了可以使用load动词加载的脚本。

```
   scripts ''
```

```
. . .
```

```
.... parts    plot     profile  scripts  stdlib ...
```

```
. . .
```

带有v参数的scripts动词列出了脚本及其完整路径。

~system表示J系统目录。

convert脚本包含几个转换实用程序。

```
   load 'convert'

   toupper 'testing 1 2 3'
```

```
TESTING 1 2 3
```

```
   tolower 'Sir Richard'
```

```
sir richard
```

第 44 章

检查点二

到了这个检查点，你应该明白：

- 作为句子来源的文本文件称为脚本文件
- 脚本文件定义全局名字
- 如何创建一个新的临时脚本文件
- 如何将临时脚本文件保存为永久文件
- 如何运行脚本文件来执行语句
- 如何在脚本文件中定义动词
- 如何定义一个动词的一元和二元模式
- =.和=:之间的区别
- 局部名字和全局名字的区别
- 区域是一组全局名字
- 可以有多个区域
- base区域是你通常使用的

通过做以下练习来检查你的理解：

- 创建一个新的临时脚本文件
- 在脚本中将square定义为使用*:对其参数进行平方的单子
- 将脚本保存在用户目录中，并命名为square.ijs
- 运行脚本并测试动词square
- 关闭J，重新启动，使用load '~/ j9.4- user/ square.ijs运行square.ijs并测试它（路径描述可能因J版本而异）

第 45 章

调试——逐步执行

在前面的一节中，你向动词定义添加了调试行，使你能够在运行动词时查看中间步骤的结果。 有时候你需要更强大的工具。

在J的最新版本中包含了调试工具，但如果没有，你可以用load 'debug'来加载调试工具。

打开脚本文件cf.ijs并运行它。

让我们执行摄氏度，但是每一行都有一个停顿，以便你可以确切地看到正在发生什么。

```
   dbss 'centigrade *:*'
```

dbss (Set Stop)的参数声明在执行所有摄氏度的行（用*:*表示）之前有一个断点。

```
   dbr 1
```

参数为1的dbr要求系统在发生错误或断点时暂停执行。当动词被暂停时，它在执行过程中停止。 你可以检查定义、更改定义，还可以恢复暂停动词的执行。

```
   centigrade 212
```

|stop

```
|       t1=.y-32

|centigrade[0]
```

错误报告（在左侧行首的竖线）指出执行在摄氏度时在第0行暂停，并显示该行中的句子。

摄氏度的执行被暂停，缩进六个空格，而不是三个，表示暂停。变量y是参数。

```
      y

212
```

暂停发生在行执行之前，所以t1没有被定义，如果你试图查看它，你将得到一个值错误。

使用dbrun继续执行。它将执行当前行，因为所有行都设置了断点，所以它将在下一行暂停。

```
      dbrun ''

|stop

|       t2=.t1*5

|centigrade[1]

      t1

180

      t1*5

900
```

现在，摄氏度在第一行暂停了，正如你所看到的，你可以检查在第0行中定

义的局部t1的值。继续调试后续的行，检查局部名字。

```
      dbrun ''
|stop
|       t3=.t2%9
|centigrade[2]
      t2
900
      t2%9
100
      dbrun ''
100
```

摄氏度执行完毕后，你又回到了正常的三个空间的缩进。
关闭调试暂停请求并重置为没有断点。

```
   dbr 0
   dbss ''
```

第 46 章

调试——处理错误

让我们在你的摄氏度动词中引入一个错误，看看它是什么样子的，以及你如何发现和修复它。

打开你的cf.ijs脚本并编辑第一行，通过在连词=.右侧的表达式两边添加引号来制造错误。

```
t1 =. 'y - 32'
```

t1不再被定义为y - 32的结果，它将被定义为字符串。

运行脚本生成新的定义。关闭调试暂停，并清除断点，然后运行有bug的摄氏度动词。

```
   dbr 0  NB. disable suspension

   dbss ''

   centigrade 212 NB. Error messages in versions prior to J9.4

|domain error

|   t2=.t1    *5


   centigrade 212    NB. Expanded error messages in J9.4
```

```
|domain error in centigrade, executing dyad *

|x is character

|   t2=.t1    *5
```

你正在禁用暂停（dbr 0）的情况下运行，所以执行没有在摄氏度上暂停，缩进依然是正常的3个空格。

如果你看一下错误行，很明显，5是乘法的有效参数，所以t1一定有问题。但是你不知道t1的值。 你可以查看源代码找错误，但是，在复杂的情况下，使用debug可能会更快。

启用暂停并重新运行。

```
   dbr 1   NB. enable suspension

   centigrade 212

|domain error

|   t2=.t1    *5

|centigrade[1]
```

此时光标会缩进6个空格表示暂停，因为摄氏度被暂停了，你可以看看t1的值。

```
      t1

y - 32
```

从t1的显示可以清楚地看出它是一个字符串，而不是所需计算的数字。现在可以查看源代码， 看看t1是在哪里定义的，并发现引号不应该出现在那里。

编辑源代码，通过删除引号来修复定义，并运行脚本来重新定义摄氏度动词。

要重新运行第0行来正确地定义t1，你可以通过使用dbjmp从第0行继续执行。

```
      dbjmp 0
```

```
100
```

由于没有设置断点，也没有其他错误，因此摄氏度第0行被执行，它将给局部名字t1设置正确的值，并继续执行直到完成。

第 47 章

比较原语

如果左右参数相等，则双子=的结果为1，如果它们不同，则结果为0。

```
   23 = 34
0
   23 = 23
1
   a =. 'd'
   a = 'c'
0
   7 + a = 'c'
7
   7 + a = 'd'
8
```

一些编程语言将比较原语（如=）的结果视为非数字的True和False值。在J中，比较的结果只是数字。

还有其他几个比较动词：小于<、小于等于<:、大于>和大于等于>:。这些比较原语有时被称为关系原语。

```
   7 < 8

1

   7 < 7

0

   7 <: 7

1
```

第 48 章

控制结构

在摄氏度中，定义中的句子只是一个接一个地按顺序执行。要有条件地控制执行哪些句子，可以使用控制结构。

控制结构由控制词和句子组成。下面是一个控制结构的例子：

```
if. x = 'c'

 do. centigrade y

 else. fahrenheit y

end.
```

控制词if.开启控制结构，控制词end.结束控制结构。x = 'c'句子的值是测试结果， 它决定了控制结构中哪个句子被执行。如果测试结果为1，则控制词do.后的句子被执行。 如果测试结果是**任何其他值**，则控制词else.后面的句子被执行。 用汉语解释就是：如果左参数x等于字母c，则执行摄氏温度，否则执行华氏温度。

使用此功能可以在cf.ijs脚本中添加一个新的动词，该动词将根据左参数的值将数字 从华氏度转换为摄氏度，或从摄氏度转换为华氏度。打开cf.ijs脚本，并在最后添加以下定义。

```
NB. convert f to c if x is 'c', otherwise c to f
```

```
convert =: dyad : 0

if. x = 'c'

 do. centigrade y

 else. fahrenheit y

end.

)
```

以上定义了动词的二元形式。这个双子有一个左参数x，一个右参数y。动词的左参数为'c'时， 将华氏度转换为摄氏度。除了'c'以外的任何左边参数都会将摄氏度转换为华氏度。

注意，动词华氏度和摄氏度的用法就像使用原语动词一样。

运行脚本并测试转换。

```
   'c' convert  212

100

   'f' convert 100

212
```

通常一个句子是脚本中的一行。但是，使用控制词将一行分隔成几个句子，就有可能在一行中包含多个句子。

下面的单行相当于前面使用的多行。

```
if. x = 'c' do. centigrade y else. fahrenheit y end.
```

控制结构只允许在脚本中使用，不能直接在REPL窗口中输入执行控制结构。

有九种控制结构模式：

```
if. T do. B end.

if. T do. B else. B1 end.

if. T do. B elseif. T1 do. B1 elseif. T2 do. B2 end.

try. B catch. B1 end.

while. T do. B end.

whilst. T do. B end.

for. T do. B end.

for_i. T do. B end.

select. T

 case. T0 do. B0

 case. T1 do. B1

 fcase.T2 do. B2

 case. T3 do. B3

end.
```

控制结构以if.、try.、while.、whilst.、for.、 for_i.或select.开始，并以一个匹配的end.结束。

以T或B开头的单词表示由0个或多个句子组成的块，并且可以包含嵌套的控制结构。

T块中最后一个句子的结果决定接下来执行哪个块，以及控制结构是否执行完毕。

通常，T块是一个简单的句子，可以做一个简单的测试，就像上面例子中的一样。

try.控制结构是一个有趣的结构。它执行B块句子，如果没有错误，它就跳到结构的末尾。但是，如果出现错误，则执行B1块。

while.控制结构执行T块，如果其结果不为0，则执行B块并继续执行，直到T块的结果为0。如果T块第一次为0，则B块不执行。

whilst.控制结构与while.相同，除了T块第一次被跳过。这意味着B块总是至少执行一次。

在for.控制结构中，B块对阵列A中的每一项求值一次，该阵列A是由求值T块产生的。在for_i.形式中， 局部名字i被设置为每次对B块求值时项的值，i_index被设置为该项的索引。控制词break.跳到for.控制结构的结尾， 控制词continue.跳到B块对下一项的进行求值。

在select.控制结构中，T的结果R与Ti的结果Ri进行比较，第一个匹配的case.或fcase.中的Bi将被执行。 如果匹配的是case.，则select.控制结构的执行就此结束；如果匹配的是fcase.，则下一个case.或fcase. 的B(i+1)块将被执行。如果下一个是fcase.，则还会继续执行下下一个，直到遇不到fcase.或执行到select.的结尾。

有关控制结构的更多信息，请参考NuVoc。

第 49 章

检查点三

到了这个检查点，你应该明白：

- 动词调试的一般方法
- 控制词如何将将句子分组成块来创建控制结构
- T块测试结果是什么
- 测试结果如何决定执行哪个B块
- 测试结果如何决定控制结构的执行何时结束

通过做以下练习来检查你的理解：

- 单步调试温度转换动词
- 创建一个临时脚本文件，并定义一个名为conv的动词，它类似于温度转换，但坚持使用'f'参数来进行华氏度的转换，并且当 做参数既不是'f'也不是'c'时，返回一个字符串表示错误。 **提示：**使用下面的控制结构示例，或者使用select.控制结构

```
if. x = 'c' do. ...

  elseif. x = 'f' do. ...

  elseif. 1 do. 'left arg not c or f'

end.
```

- 创建一个临时脚本文件，并定义一个名为plus的双子，将其左参数和右参数相加。但是，如果有错误，它应该给出一个字符串结果。

提示：

- 使用dyad : 0；
- 4 plus 9应当返回13；
- 'a' plus 9应当返回错误字符串（比如，'there was an error'）；
- 使用try.控制结构来捕捉错误并给出字符串结果。

第 50 章

列表相加的基础方式

你可以创建数字列表。

```
   a =. 12 24 47

   b =. 12 34 45
```

如果你想在Basic之类的语言中把两个数字列表相加，你必须依次从每个列表中获取每个数字，将它们加在一起， 然后将这个新结果添加到结果列表的末尾。

要以这种方式将两个数字列表相加，你需要一些新的原语。你需要一种从列表中获取一个数字的方法。动词{ (from)可以做到这一点。

```
   0 { 7 9 2 4   NB. index 0 value

7

   1 { 7 9 2 4  NB. index 1

9

   2 { 7 9 2 4  NB. index 2

2
```

```
   3 { 7 9 2 4  NB. index 3
4
```

你需要能够向结果列表添加一个新的结果值。可以使用动词,。

```
   7 9 2 , 4 NB. append 4 to the list
7 9 2 4
   7 9 2 4 , 7  NB. append 7 to the list
7 9 2 4 7
   a =. 7 9 2 4
   a =. a , 23
   a
7 9 2 4 23
```

你需要知道列表中有多少个数字，这样你就知道什么时候完成了。单子#(tally)告诉我们列表中有多少个数字。

```
   # 7 9 2 4
4
   # 7 9 2 4 7
5
```

你还需要方法来创建一个空结果，你将向其中添加每个新结果。空字符串可用来创建空列表。

```
   r =. ''      NB. an empty string
```

有了这些新的动词，再加上你已经知道的，就可以编写一个Basic或Java风格的程序来把两个列表相加。

创建一个临时脚本文件并加入addlists的定义。

```
addlists =: dyad : 0

r =. ''

count =. # x

i =. 0

while. i < count do.

 left =. i { x

 right =. i { y

 sum =. left + right

 r =. r , sum

 i =. i + 1

end.

r

)
```

局部名字i是从每个列表中选择数字的索引。它从0开始，从左参数和右参数中选择第一个数字。 在while.控制结构的最后，i.自增加1，以便下次执行块时它将选择下一个数字。while.控制结构测试 以查看i是否小于列表参数的长度。当i等于要加在一起的列表的长度时，控制结构完成。 局部名字left和right被定义为下一对数字。它们被加在一起，并添加到结果r的末尾。

运行脚本并测试addlist的定义。

```
   2 3 4 addlists 4 5 6

6 8 10
```

如果你在定义中打错字，就会得到一个运行错误或错误的答案。在这种情况下，你应该仔细检查是否输入了正确的定义。

某些错误（例如省略了更新i值的行）会造成while.死循环。因为while.永远不会结束，程序继续添加左和右参数的第一个元素，永远不会处理下一个元素。如果你处于这样的循环中，则需要运行jbreak程序来中断J的执行。在Windows或Mac中jbreak是**黄色的J图标**，Unix中是**jbreak shell脚本**。

事实上，值得一看死循环会发生什么。首先，确保你知道如何运行jbreak程序来发出中断信号（在Windows或Mac中：找到黄色的J图标，在Unix中打开一个命令窗口，准备在J目录中运行jbreak shell脚本）。在J空闲时运行jbreak不会做任何事情。编辑addlists定义，使其不更新i的最简单方法是在i =. i + 1句子前面加上NB.。运行脚本并测试动词。它将一直运行，直到你使用jbreak中断执行（或取消J任务）。

对于这样一个简单的事情，这个用了11行的定义似乎过于冗长。该定义可以通过组合句子来简化。在临时脚本文件中创建定义的第二个版本。

```
adda =: dyad : 0

r =. ''

count =. # x

i =. 0

while. i < count do.

 r =. r , (i { x) + (i { y)

 i =. i + 1

end.
```

```
r

)
```

运行脚本并测试这个新版本。

```
   2 3 4 adda 4 5 6

6 8 10
```

这基本上是大多数编程语言的程序员如何把两个数字列表相加的方法。该程序可以进一步精简，但它仍然必须是一个控制结构， 每次处理一个数字。大多数语言只知道如何将两个单个数字相加。要把两个数字列表加起来，你需要编写一个控制结构， 循环并显式地依次把两个列表中的每个元素相加。

第 51 章

列表相加的J方式

J知道如何处理单个数字，但它也知道如何处理列表。

因为J知道如何进行列表相加，所以可以编写第三个更简单的版本。

```
addb =: 4 : 'x + y'
```

将此定义添加到临时脚本文件中，运行脚本并对其进行测试。

```
   2 3 4 addb 4 5 6
```

```
6 8 10
```

现在，你可能意识到addb是如此简单，以至于没有必要定义它。

```
   2 3 4 + 4 5 6
```

```
6 8 10
```

在J中，你可以直接把两个数字列表相加，因为动词+知道所有关于数字列表的信息。

多年来的研究和思考已经渗入J动词与列表一起工作的方式。例如，如果你想对列表中的每个数字加1。

```
   1 adda 2 3 4
```

```
3
```

基本方式的动词adda给出了答案，但这是错误的答案。事情是这样的。while.使用左参数的长度（为1）来确定在结果中应有多少个元素，因此只计算第一个结果。

```
   2 3 4 adda 1
```

```
|index error
```

```
|   r=.r,(i{x.)+(i    {y.)
```

上面的代码会产生一个错误，因为while.使用长度3（左参数中的数字个数）但是右参数没有那么多数字，所以你会得到一个错误。

但是J动词+以你喜欢和期望的方式处理这两种情况！

```
   1 + 2 3 4
```

```
3 4 5
```

```
   2 3 4 + 1
```

```
3 4 5
```

谢天谢地，动词addlist和adda都在一个临时文件中，很容易删除，因为在J中显然不需要它们。现在关闭并删除那个临时脚本。

操纵列表而不是单个元素，这个简单概念在整个J中得到了扩展。

```
   2 4 6 * 7 8 9
```

```
14 32 54
```

```
   2 * 2 3 4
```

```
4 6 8
```

```
   2 3 4 * 2
4 6 8
   5 6 7 % 2
2.5 3 3.5
   2 3 4 - 3
_1 0 1
```

它也适用于比较原语。

```
   2 3 4 = 7 3 8
0 1 0
   2 < 0 1 2 3 4 5
0 0 0 1 1 1
   't' = 'testing'
1 0 0 1 0 0 0
```

如果它适用于原语，那么像摄氏度这样的动词呢？

```
   fahrenheit _40 0 100
_40 32 212
```

适用于二元转换吗？

```
   'c' convert _40 32 212
_40 0 100
```

```
   'f' convert _40 0 32 100 212

_40 32 89.6 212 413.6
```

动词的拓展使其能够一致地操作列表，这非常非常强大，也是J与大多数其他语言的显著区别。 重要的是，你要把这一点融入到思考解决问题的方式中。

第 52 章

更多原语

单子i. (integers)的结果是从0到其右参数的整数列表。

```
   i. 2
0 1

   i. 4
0 1 2 3

   i. 6
0 1 2 3 4 5
```

双子$成为赋形。

```
   5 $ 7 NB. a list of 5 7's
7 7 7 7 7

   8 $ 23   NB. a list of 8 23's
23 23 23 23 23 23 23 23
```

单子? (roll)在范围0到参数减1内生成一个随机数。参数0则生成一个大于等于0且小于1的随机浮点数。 生成结果取决于掷骰子的方式。

```
   ? 10

6

   ? 10

0

   ? 10 10 10 10  NB. 4 numbers in range 0 to 9

3 0 4 6

   ? 5 $ 100      NB. 5 numbers in range 0 to 99

58 93 84 52 9

   ? 0 0 0

0.708477 0.553165 0.00766673
```

双子^ (power)的结果是左参数的幂，幂次数由右参数给出。

```
   3 ^ 2  NB. 3 * 3

9

   2 ^  3   NB. 2 * 2 * 2

8

   2 ^ 4    NB. 2 * 2 * 2 * 2

16

   2 2 2 2 2 2 ^ 0 1 2 3 4 5
```

```
1 2 4 8 16 32

   2 ^ 0 1 2 3 4 5

1 2 4 8 16 32

   2 ^ i. 6

1 2 4 8 16 32
```

二元动词o. (circle)是字母o加上.的变形，它提供了圆（三角）函数。特别地，左参数为1时，动词o.求右参数的正弦值。

```
   1 o. i.7

0 0.841471 0.909297 0.14112 _0.756802 _0.958924 _0.279415
```

单纯语言描述很难搞明白这些动词的含义，所以下一章我们将用图表来展示这些数据。

第 53 章

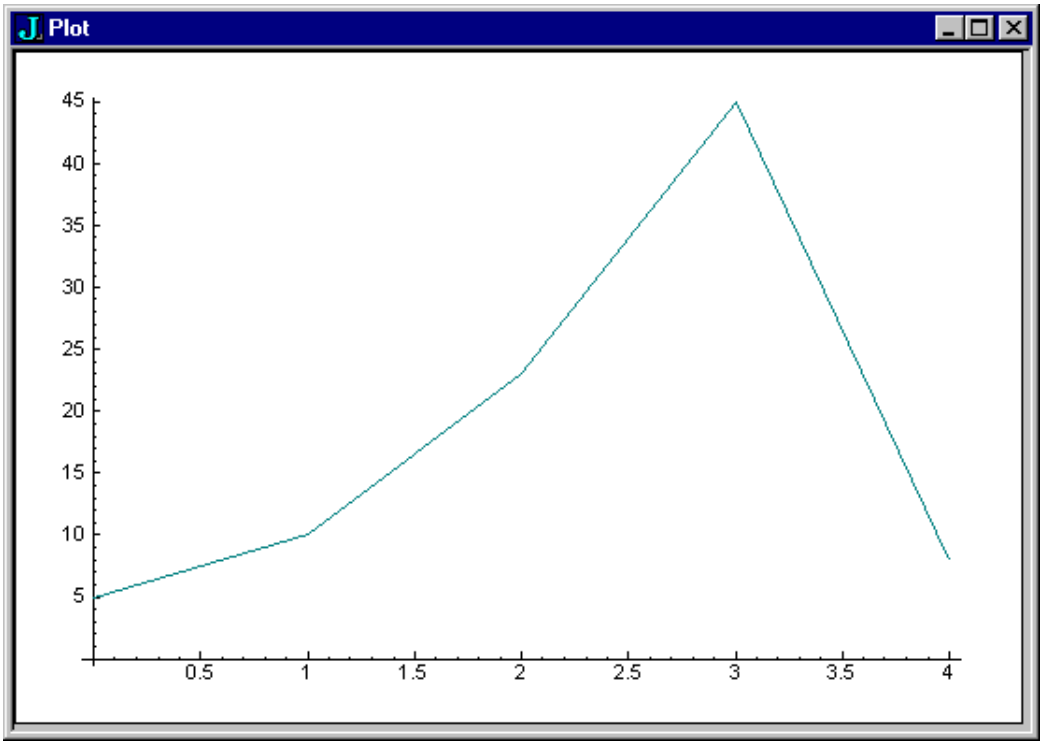
绘图

要使用绘图功能，你需要加载它。

```
load 'plot'
```

尝试一个简单的绘图。

```
plot 5 10 23 45 8
```



绘图位于一个单独的窗口。关闭绘图窗口，丢弃它。

现在可以绘制数据了，让我们看一下在前一节中使用的一些原语。

```
plot 2 ^ i. 5
```

```
plot ? 5 $ 100
```

```
plot ? 50 $ 100
```

```
plot ? 100 $ 100
```

```
plot ? 1 + i. 50
```

```
plot ? 1 + i. 100
```

通过左参数定制绘图。

```
'TITLE myplot;TYPE bar' plot 2 ^ i. 5
```

或者试试之前计算的正弦值。

```
plot 1 o. i. 16
```

在trig.ijs脚本中定义了一系列实用程序，你可以在这里使用。使用load来加载该脚本。

```
load 'trig'

plot sin i. 16
```

为了生成更精细的图，你需要在类似的范围内提供更多的数值结果。

```
plot sin 0.2 * i.60

plot cos 0.2 * i.60
```

第 54 章

绘图区域

绘图还说明了使用区域的一种常用技术。关闭J并重新启动它以获得一个干净的环境。 base区域中没有动词名字，但是profile.ijs脚本加载了一些其他标准区域。

```
   names 3 NB. verbs

   names 6  NB. locales
```

```
base     j        jcompare jgl2     jp       jqtide   jregex
 ↪  jrx      jtask    z
```

动词load（在z区域中定义）将plot.ijs脚本加载到plot区域中。

```
   load 'plot'

   plot ? 5 $ 100

   names 3

   names 6
```

```
base  j     jcfg  jplot z
```

```
   plot
```

```
+----------+

|plot_plot_|

+----------+
```

动词plot没有在base区域中定义，而是在z区域中定义。当它在base区域中执行时， 来自z区域的定义就像在其定义在base区域中一样执行。输入plot名字将显示其定义。 关于plot定义的有趣之处在于，它执行在plot区域中定义的plot动词。 z区域中的plot是一个在所有语言区域中都有效的覆盖，它的作用是在执行plot区域中的plot动词。

> 注：在较新的J版本中（J9.7以上），plot.jis已经被替换为别的脚本，具体看J实现。

这种技术将像plot包这样的工具加载到它自己的区域中，然后在z区域中定义覆盖动词， 以便用简单名字引用（不需要加区域后缀）调用功能区域中所需的动词。可以将plot区域中的定义 视为该工具包的私有实现，而通过在z区域中定义而公开的名字是公共或发布的接口。

第 55 章

打印精度

打印精度是显示数字时显示的位数。

```
   1 % 3
```

```
0.333333
```

```
   10 % 3
```

```
3.33333
```

```
   100 % 3
```

```
33.3333
```

使用列表和一些新的原语，现在可以更简洁地看明白打印精度：

```
   (10 ^ i. 6) % 3
```

```
0.333333 3.33333 33.3333 333.333 3333.33 33333.3
```

你可以猜到默认的打印精度是6，因为在每种情况下显示6位数字。

```
   a =. 1e5 + 10 % 3
```

```
   b =. a + 0.1

   a

100003

   b

100003

   a = b

0
```

如果只显示6个数字，a和b看起来是一样的，尽管它们并不相等。在这种情况下，你需要看到更多的数字。 使用下面的动词可以改变打印精度。

```
   pps 9

   a

100003.333

   b

100003.433
```

打印精度设为9，因此有足够的细节可以看到发生了什么。

```
   pps 6

   %3 9 13

0.333333 0.111111 0.0769231

   pps 12

   %3 9 13
```

```
0.333333333333 0.111111111111 0.0769230769231
```

默认的打印精度6对于大多数情况来说是足够的，因为你通常不需要看到所有这些额外的数字细节。 然而，重要的是要知道额外的数字确实存在，只是为了方便起见，显示被缩短了。

第 56 章

不精确数值

数值在计算机中的存储方式限制了数值的最大位数。这个最大值取决于硬件， 但通常执行以20为参数的pps动词将保证显示一个数值的所有可用数字。

```
   pps 6

   %3

0.333333

   pps 12

   %3

0.333333333333

   pps 20

   %3

0.33333333333333331483
```

在精度为20位的情况下，%3的结果以17位数字显示该数值的所有细节。%3的结果不是精确的数学结果，但它是可以存储在计算机中的最接近精确

结果的数字。 你从精确数学中得到的结果和计算机中数值存储方式的限制之间的差异可能会令人困惑。

%3没有精确地存储在计算机中，如果你意识到精确的十进制表示需要无数个3，那么你可能不会感到太惊讶。

由于计算机内部以二进制格式存储数值（其中每个数字都是0或1），而不是你熟悉的十进制格式（其中数字的值范围从0到9）， 因此还有一个额外的困惑来源。这样做的结果是，即使是非常简单的十进制数，在转换成计算机的二进制格式时， 也会被存储为一个不精确的值。

```
   pps 20

   0.5 0.25 0.1

0.5 0.25 0.10000000000000000555
```

0.5和0.25被精确地存储，但是0.1被不精确地存储，当以最大精度显示时显示为0.10000000000000000555。

这些都是计算机存储浮点数时无法该变的事实，适用于所有计算机语言，而不仅仅是J语言。 通常你可以忽略这些细节，但如果你不知道发生了什么，它们有时会引起问题或困惑。

第 57 章

容差

这一部分有点高级，理解它并不重要。如果能理解，那很好。如果不明白，不要担心，直接进入下一部分。

对于使用浮点数的某些工作，本节非常重要，同时还需要更详细地了解硬件如何存储和操作数值。然而，对于大多数工作，这部分可以忽略。

在上一节的最后，我们看到计算机不能总是精确地表示数值。那么如果它们不能被精确地表示出来呢，我们怎么知道一个数是否等于另一个数？ 解决这个问题的方法是宽容比较的概念。也就是说，数值不必完全相同才能被认为是相等的。

让我们通过实验来了解比较的宽容度如何，方法是逐渐使值远离100。使用输入行召回快捷键Shift+Ctrl+Up对于这类操作非常有用。

```
   100 = 100
1
   100 = 99.999999999999986
1
   100 = 99.99999999999998
1
```

```
   100 = 99.9999999999999

1

   100 = 99.999999999999

1

   100 = 99.99999999999

0
```

在上面的示例中，你越过了这条线，使得数值离100足够远，以至于它们不再被认为是相等的。让我们看另一个例子。

```
   a =. 23

   b =. a - 1e_12

   c =. a - 1e_11

   a

23

   a-b

9.9831254374294076115e_13

   a=b

1

   a-c

1.0000889005823410116e_11
```

```
a=c
```

```
0
```

a和b的值足够接近，可以认为是相等的。a和c的值不够接近，不能认为是相等的。 足够接近和两个数值之间的差异相关。在这两种情况下，差异都很小，但b比c更接近a。 阅读NuVoc对=定义， 你将看到足够接近和不够接近之间的分界线是由较大的数值乘以默认容差值2^_44的结果决定的。 也就是说，足够接近与否与数值的大小有关。

将两个差异值和容差值进行比较：

```
(a - b , c) <: tolerance    NB. Remember, this is
↪  equivalent to (a - (b,c)) due to right to left
↪  precedence.
```

```
1 0
```

a和b之间的差异小于或等于容差，而a和c之间的差异则不是。

第 58 章

检查点四

到了这个检查点，你应该明白：

- 原语适用于列表
- 你的自定义动词适用于列表
- 如何使用几个新动词
- 如何使用绘图功能
- 比较原语，如=，给出数值0和1作为结果

通过做以下练习来检查你的理解：

- 查找NuVoc中动词i. (integers)、$ (shape)、? (roll)、 ^ (power)和o. (circle)的定义；本指南在大多数情况下，只介绍了它们的一部分功能，因此你必须仔细阅读定义， 以便能够忽略尚未介绍的部分，并挑选出已经介绍的部分
- 用这些新的原语进行实验

第 59 章

原子

由单独的实体构成的名词，是一个原子。

```
23

'a'

b =. 23

a =. 'q'
```

一个原子的维度为0。

第 60 章

列表

由一列原子构成的名词就是列表。

```
23 24 25

'this is a string'

b =. 7 14 21

a =. 'another string'
```

列表的维度为1。维度的长度是该维度中原子的数量。

第 61 章

表格

表格是由原子组成的二维阵列。表格不能像原子或列表那样直接写成字面量，而是必须用原语创建。 双子$ (shape)可以创建表格。左边的参数表示每个维度中的元素数量，右边的参数提供用于填充表格的元素。

```
   2 3 $ 7 NB. a 2 by 3 table of 7's
7 7 7
7 7 7
   2 3 $ 7 8 9 10 11 12
 7  8  9
10 11 12
   2 3 $ 7 8    NB. cycle through atoms to get enough
7 8 7
8 7 8
   3 4 $ 'abcdefghijklmnopqrstuv'
```

```
abcd

efgh

ijkl
```

单子$ (shape of)给出了它的参数的形状。形状是参数每个维度上元素数量的列表。

```
   a =. 2 3 $ 7

   $ a
2 3
```

前面介绍的用于创建整数列表的单子i. (integers)可用于创建整数表格。

```
   i.5        NB. list of 5 integers
0 1 2 3 4

   i. 2 3   NB. 2 by 3 table of integers
0 1 2

3 4 5
```

第 62 章

阵列

原子、列表和表都是阵列。J中的所有名词都是阵列。原子有0个维度，列表有1个维度，表有2个维度。 这可以扩展到更高维度的阵列。

前面讨论的关于列表和表的原语$也适用于高维阵列。

```
   2 3 4 $ i. 24

 0  1  2  3

 4  5  6  7

 8  9 10 11


12 13 14 15

16 17 18 19

20 21 22 23
```

上面是一个三维阵列。空行表示第一维与第二维和第三维之间的分隔。

```
   b =. 2 3 4 $ 'abcdef'

   b
```

```
abcd

efab

cdef


abcd

efab

cdef
```

```
   $ b
2 3 4
```

单子i.也适用于高维阵列。

```
   i. 2 3 4
 0  1  2  3
 4  5  6  7
 8  9 10 11


12 13 14 15
16 17 18 19
20 21 22 23
```

J术语中的原子、列表和表格类似于数学术语中的标量、向量和矩阵。

第 63 章

轴

比起属于“维度”，我们更倾向于使用术语“轴”。原子有0个轴，列表有1个轴，表格有2个轴。

第 64 章

形

名词的形是它每个轴上的原子数量的列表。单子$给出名词的形。

```
   a =. 4 2 3 $ 'abcdef'

   $ a
4 2 3

   $ 4      NB. atom has 0 axes and its shape is empty

   $ i. 5
5
```

第 65 章

秩

名词的秩是它的轴数。原子的秩为0，列表的秩为1，表格的秩为2，有5个轴的阵列的秩为5。 阵列的秩非常重要，它在很大程度上决定了动词如何操作阵列。

阵列的形是一个列表，其中的数字个数与阵列的轴数一样多。这意味着阵列的形的长度就是它的秩。

```
   # $ 4  NB. atom has rank 0
0

   # $ 4 5 6    NB. list has rank 1
1

   # $ 2 3 $ 'a'    NB. table has rank 2
2
```

第 66 章

空阵列

如果阵列中没有原子，那么它就是空的。一个空阵列的形是0。

```
   a =. 0 $ 0  NB. empty list

   $a
0

   $ '' NB. empty list
0

   b =. 2 0 3 $ 'a' NB. empty rank 3 array

   $b
2 0 3
```

空阵列没有原子来显示，如果不知道该期望什么，那么显示可能会让人困惑。 空阵列显示为空白行。一个0行的表格被当作0行来显示（即没有显示）；一个有3行0列的表格，显示为3个空白行。

```
   2 $ 5
```

```
5 5
   1 $ 5
5
   0 $ 5    NB. empty list displays as blank line
   2 2 $ 5
5 5
5 5
   1 2 $ 5
5 5
   2 0 $ 5  NB. 2 rows displays 2 lines


   0 2 $ 5  NB. 0 rows displays 0 lines
```

列表和表格的显示可以看起来是一样的，你必须查看它们的形来区分它们。

```
   a =. 2 $ 5
   b =. 1 2 $ 5
   a
5 5
   b        NB. b displays the same as a
5 5
```

```
   $ a
2
   $ b      NB. but b has a different shape
1 2
```

第 67 章

单原子阵列

只有一个原子的阵列称为单例。具有相同原子的所有单例都以相同的方式显示。 然而，它们有不同的秩这一事实影响了动词对它们的作用。对于初学者来说，这可能是一个陷阱。 重要的是要记住，如果它显示为原子，但行为不像原子，那么检查它的秩。

```
   atom =. 5

   list =. 1 $ 5

   atom

5

   list NB. list looks like atom

5

   atom + 23 23 23

28 28 28

   list + 23 23 23  NB. but does not behave like atom

|length error
```

```
|   list    +23 23 23
   # $ atom NB. rank of atom is 0
0
   # $ list NB. rank of list is 1
1
```

第 68 章

动词参数

J的很大一部分功能在于动词将其参数视为一系列部分的能力。 动词将自己应用于每个部分，创建一系列部分结果，然后将部分结果组装成最终结果。 接下来的几节将详细描述它的工作原理以及使用它可以做些什么。

让我们看几个例子，以了解后续学习方向。

```
   m =. i. 2 2

   m
0 1

2 3
```

你可以将具有相同秩和形的阵列加在一起。

```
   m + 2 2 $ 10 11 12 13
10 12

14 16
```

也可以把单个数值和阵列相加。

```
  10 + m
```

```
10 11

12 13
```

如果你想给第一行加上一个数值，给第二行加上另一个数值呢？

```
   10 20 + m

10 11

22 23
```

但是如果你想把这些数值加到列上呢？那你必须明确想要加到列上而不是行上。

```
   10 20 +"1 m

10 21

12 23
```

第 69 章

框架和单元格

到目前为止，名词一直被视为整体。但是，将阵列视为由单元格组成是有用的， 单元格是阵列的一部分（子阵列），当将其放置在框架中时，就构成了整个阵列。

```
   a =. 2 3 $ 'abcdef'

   a

abc

def
```

阵列a可以被认为有6个单元格，每个单元格是一个原子。框架的形是2 3， 将6个单独的单元格组装到阵列a中。视觉上（单元格是原子，框架是2 3）：

```
 a       cell 0

    b       cell 1

    ...

    f       cell 5
```

阵列a也可以被认为有2个单元格，其中每个单元格是一个列表。框架的形是2， 将单元格组装到阵列a中。视觉上（单元格是列表，框架是2）：

```
abc   cell 0

   def   cell 1
```

最后，可以认为阵列a有一个单元格，其中单元格是一个表格。框架的形为空， 将单元格组装到阵列a中。视觉上（单元格是表格，框架是空的）：

```
  abc   cell 0

   def
```

形为2 3的表格可以理解为：

- 一个以原子为单元格，形为2 3的框架
- 一个以列表（形为3）为单元格，形为2的框架
- 一个以表格（形为2 3）为单元格的空框架

类似地，形为4 3 2的阵列可以被认为是：

- 一个以原子为单元格，形为4 3 2的框架
- 一个以列表（形为2）为单元格，形为4 3的框架
- 一个以表格（形为3 2）为单元格，形为4的框架
- 一个以秩为3的阵列（形为4 3 2）为单元格的空框架

框架是阵列形的前缀。它可以是整个形（以整个形作为前缀），在这种情况下，单元格是原子。 它可以为空（空前缀），在这种情况下，以整个阵列作为单一单元格。或者介于两者之间。

单元格的形是去掉框架前缀后阵列的形。单元格形的长度就是单元格的秩。

阵列的单元格是子阵列，当它们组装到相应的框架中时，就创建了整个阵列。

第 70 章

项

阵列通常被视为具有长度为1的框架。以这种框架看阵列，其单元格的秩比阵列的秩小1。这些单元格是阵列的**项**。

列表中的项是列表中的原子。

表格中的项是表格中的行。

秩为3的阵列的项是阵列中的表格。

阵列是它的项的列表。

原子也有一个项，是它自己。

名词的计数（`# (tally)`）是名词中项的数量。

```
   # 23
1
   # 1 $ 5
1
   # i. 5
5
```

```
# i.2 3
```

```
2
```

第 71 章

k元格

秩为k的单元格也称为k秩单元格或k元格。0元格是一个原子，1元格是一个列表，2元格是一个表格，以此类推。 如果一个名词的单元格的秩是已知的，那么它的框架就是它的形除去单元格的秩所剩下的东西。

也可以使用负数，如_2元格和_1元格；这种单元格的框架的长度由其负数秩的绝对值表示。你之前已经见过_1元格：它们是项。

```
abc =. 4 3 2 $ i. 24
```

名词abc可以理解为：

- 4 3 2框架装着0元格
- 4 3框架装着1元格
- 4框架装着2元格
- 空框架装着3元格

更通用的表达方式是：

- 秩为3的框架装着0元格
- 秩为2的框架装着1元格
- 秩为1的框架装着2元格
- 秩为0的框架装着3元格

第 72 章

动词的秩

动词也有秩，决定了它如何应用于其参数。秩为k的单子应用于其参数的k元格。 左秩kl和右秩kr的双子应用于左参数的kl元格和右参数的kr元格。 动词的秩是一个强大的工具，它控制着动词应用于阵列的方式。

原语动词的秩在NuVoc定义中给出。例如，查找+的定义。秩信息紧挨着标题中的动词名称。 对于+，显示为0 Conjugate，这表明单子+(Conjugate)适用于原子。 双子+ (Plus)的秩是Plus 0 0，表示它适用于左参数和右参数的每个原子。

让我们看看把两个表相加时它是如何工作的。

```
   a =. i. 2 3

   b =. 6 + a

   a
0 1 2

3 4 5

   b
6 7 8
```

```
9 10 11

   a + b

 6  8 10

12 14 16
```

双子+的左秩是0。这意味着它应用于左参数的原子。类似地，右秩为0，也应用于其右参数的原子。 动词从其左参数中获取一个原子，从其右参数中获取一个原子，并将它们相加以创建部分结果。 它对来自左右参数的每个原子执行此操作，并创建适当数量的部分结果，然后将其组装到结果框架中以创建最终结果。

在上面的例子中，动词+的左秩为0。这意味着左参数被看作是一个装着原子的2 3框架。 类似地，右秩为0意味着右参数被视为装着原子的2 3框架。

结果的框架是由参数的框架决定的，所以它的框架也是2 3，其每个单元格都是左参数中的一个原子和右参数中的一个原子相加的结果。

第 73 章

秩协商

对于双子，动词的左秩和左参数的秩决定了左参数的框架。同样，动词的右秩和右参数的秩决定了右参数的框架。 如果左右框架相同，则每个参数中有相同数量的单元格，只需依次从左右参数中取出每个单元格，应用动词，并将结果放入结果框架中。

```
   a =. i. 2 3

   b =. 2 3 $ 7

   a + b

 7  8  9

10 11 12
```

可视化地看，左边的每个原子是如何与右边的相应原子是这么一起使用的。

```
0 1 2   +   7 7 7   gives   7  8  9

3 4 5       7 7 7           10 11 12
```

你也看到了下面的也能正常运算。

```
   a + 7

 7  8  9

10 11 12
```

可视化地看，左边的每个原子是如何与右边的相应原子是这么一起使用的。

```
0 1 2   +   7 ...       gives   7  8  9

3 4 5       ...                 10 11 12
```

这个...指示重复单元格以提供所需的参数。这个7右边及下边的...右边表示7在两个轴上重复。

但是下面这个呢？

```
   a + 3 4

3 4 5

7 8 9
```

可以再次看到，右参数的单元格如何重复以提供所需的动词参数。

```
0 1 2   +   3 ...       gives    3 4 5

3 4 5       4 ...                7 8 9
```

但是参数的单元格一定要能协商一致。

```
   a + 3 4 5

|length error

|   a    +3 4 5
```

可视化地看，情况是这样的：

```
0 1 2   +   3 ...      gives    3 4 5

3 4 5       4 ...               7 8 9

            5 ...   error - ran out of lefts
```

上面的例子很简单，但是考虑下面秩为3的名词。

```
   b =. i. 2 3 4

   b + a

 0  1  2  3

 5  6  7  8

10 11 12 13


15 16 17 18

20 21 22 23

25 26 27 28
```

这个可视化起来比较复杂。

```
 0  1  2  3   +   0 ...   gives   0  1  2  3

 4  5  6  7       1 ...           5  6  7  8

 8  9 10 11       2 ...          10 11 12 13


12 13 14 15       3 ...          15 16 17 18

16 17 18 19       4 ...          20 21 22 23
```

```
20 21 22 23       5 ...          25 26 27 28
```

类似地：

```
   b + 2 3

 2  3  4  5

 6  7  8  9

10 11 12 13


15 16 17 18

19 20 21 22

23 24 25 26
```

可视化：

```
 0  1  2  3   +   2 ...   gives  2  3  4  5

 4  5  6  7       ...            6  7  8  9

 8  9 10 11                     10 11 12 13


12 13 14 15       3 ...          15 16 17 18

16 17 18 19       ...            19 20 21 22

20 21 22 23                      23 24 25 26
```

秩协商规则很简单。如果左右框架是一样的，那就没有问题。 否则，一个框架必须是另一个框架的前缀，并且其单元格要按照另一个框架的剩余轴结构进行重复， 以提供所需的参数。

第 74 章

秩连词 ”

原语"（双引号，不是两个单引号）是秩连词。连词还没有介绍，后面的部分会有更多的细节。 现在，只要把连词想象成类似于双子动词，因为它有一个左参数和一个右的参数，并生成一个结果。 这里"的特殊用法是，左参数是动词，右参数是名词。是的，连词可以是动词形参，也可以是名词形参， 而动词只能是名词形参。

在名字章节有一个例子，你直接将名字定义为动词。

```
plus =. +
```

这种定义方式比用来定义摄氏度的定义方式更直接。它被称为**隐式**定义，将在后面的小节中进行更详细的讨论。 名称plus被定义为原语+，因此具有与+的单子（秩为0）和双子（秩为0 0）有相同的秩。

秩连词从其左参数产生一个新的动词，新动词使用从其右参数获得秩信息。

```
plus000 =. + " 0 0 0
```

"的右参数是NuVoc中给出的原语+的秩信息（查找+并记录名称中的秩信息）。 第一个0是单子参数的秩。第二个和第三个0分别是双子左和右参数的秩。

由于plus000和原语+具有相同的秩的+，因此它的行为应该与+或plus一样。 你可以从前面关于秩协商的章节中借用一些实验来验证这一点。

```
   a =. i. 2 3

   a plus000 a

0 2  4

6 8 10

   a plus000 1 2 3

|length error

|   a   plus000 1 2 3
```

出现长度错误是因为参数与前一节不一致。左框架为2 3，右框架为3 3，不是2 3的前缀； 左参数有额外的单元格，但是右参数没有相应的单元格。

然而，将列表1 2 3加到左参中的每个列表似乎是合理的。你知道你想让它做什么。视觉上:

```
0 1 2   +   1 2 3      gives   1 3 5

3 4 5       ...                4 6 8
```

你想要一个+的变体，将左参数的列表和右参数的列表加起来。 可以通过修改连词"的参数更来实现这一点，以表明双子的左右秩是列表。

```
   plus011 =. + " 0 1 1

   a plus011 1 2 3

1 3 5

4 6 8

   1 2 3 plus011 a

1 3 5
```

```
4 6 8
```

在实践中，你不必费心为+的这种特定应用程序命名，而是直接使用表达式。

```
   1 2 3 +" 0 1 1 a
1 3 5

4 6 8
```

由于+是直接应用的，而且两个参数的秩都是1，所以可以使用+"1这种较短形式，即所有参数的秩为1。

```
   1 2 3 +"1 a
1 3 5

4 6 8
```

在本例中，左框架为空，单元格形为3；右框架为2，单元格形为3。空是2的前缀，所以框架一致的。

有一件事你必须知道。

```
   a +"1 1 2 3
|length error

|   a    +"1 1 2 3
```

问题是J不知道你想要第一个1作为参数而第二个1是字面量1 2 3的一部分。 发生的情况是，字面量1 1 2 3被用作"的正确参数，而"被定义为只允许参数为1 2或3个数字， 因此存在长度错误。你需要让J知道第一个1属于"，而1 2 3是一个字面量。可以通过使用括号来明确这一点。

```
   a (+"1) 1 2 3
```

```
1 3 5

4 6 8
```

```
   a +"1 (1 2 3)
```

```
1 3 5

4 6 8
```

第 75 章

结果的形

在前面几节中，结果的形这个问题被掩盖了。对于单子，结果的框架与参数的框架相同。 对于双子，结果的框架是参数的框架中较长的框架（或者如果它们相同，则任意一个框架）。

像+这样的动词，每个原子参数都有一个原子结果，这很简单。行为更复杂的动词会让事情变得更有趣。

考虑动词$。在NuVoc中查找它，你会看到它对于单子的秩为_， 这表示无限（无界）秩，并且意味着动词适用于整个参数。如果你认为单子$的结果的形是其整个右参数的形，这说得通。 双子的左秩是1，这意味着它适用于左参数的列表。双子的右秩是无界的_，因此适用于整个右参数。

```
   2 4 $ i.3

0 1 2 0

1 2 0 1

   2 4 $"1 0 i.3

0 0 0 0

0 0 0 0
```

```
1 1 1 1

1 1 1 1


2 2 2 2

2 2 2 2
```

第一个例子是你之前见过的，但第二个例子是什么呢？$"1 0表示$将以左侧列表（1元格）和右侧原子（0元格）的形式获取单元格参数。 左框架是空（在取了一个1元格后，左参数形的左边没有任何东西），右框架是3（右参数中有3个0元格）。所以结果的框架是3。

```
2 4 $ 0   gives   0 0 0 0   left 1-cell $ right 0-cell

                  0 0 0 0


... $ 1   gives   1 1 1 1   repeat 1-cell $ next 0-cell

                  1 1 1 1


... $ 2   gives   2 2 2 2   repeat 1-cell $ next 0-cell

                  2 2 2 2
```

结果的框架是3，框架里的东西是2 x 4的表格，所以最终结果的形是3 2 4。

```
   $ 2 4 $"1 0 i.3

3 2 4
```

秩（名词秩、动词秩、框架、单元格和秩连词）适用于所有动词，并且大大增加了你使用任何动词的方式。

第 76 章

检查点五

到了这个检查点，你应该明白：

- 术语原子、列表、表、阵列、轴、秩、形、项、框架和单元格
- 名词的秩
- 动词的秩
- ”（秩连词）
- 秩协商
- 秩如何决定动词应用于哪个单元格参数
- 结果是如何由部分结果组成的

通过做以下练习来检查你的理解：

- 尝试使用你知道的原语，使用"将它们应用于单元格，并查看部分结果如何构建最终结果
- 不要把你的实验局限于像+这样的动词，也可以尝试像$ (shape)和, (append)这样的动词。

第 77 章

副词

副词与动词相似，但在以下方面有所不同：

- 副词只有一个左参数（动词具有双重性，要么有一个右参数，要么有一个左参数和一个右参数）
- 副词可用于名词或动词（动词仅用于名词）
- 副词通常生成一个动词作为结果（动词总是有一个名词结果）。

副词生成的动词结果称为派生动词。

原语/是副词。它的结果是一个新的动词。如果使用派生动词的一元形式，则将/称为插入。 如果使用派生动词的二元形式，则将/称为表格化。

第 78 章

插入副词

如果将/应用于动词，然后使用派生动词的一元模式，则将其称为插入。派生动词通过在参数的项之间插入原始动词的方式应用其一元模式。

```
   sumover =. +/
```

副词/接受其左侧的动词参数+，并创建一个名为sumover的新动词。

```
   sumover 7 5 10
```

```
22
```

参数7 5 10的项是3个原子7，5和10，sumover的定义是在参数的项之间插入原来的动词。

```
   7 + 5 + 10
```

```
22
```

```
   sumover i. 8
```

```
28
```

如果你对表格这么做呢？

```
   a =. i. 2 3

   a

0 1 2

3 4 5

   sumover a

3 5 7
```

这很有趣，但让我们仔细看看。a的项是两个列表0 1 2和3 4 5。 动词summover被定义为将+（/的原始参数）放在其参数的项之间。

```
   0 1 2 + 3 4 5

3 5 7
```

如果有更多的行呢？

```
   a=. i. 3 4

   a

0 1  2  3

4 5  6  7

8 9 10 11

   sumover a

12 15 18 21
```

a的项是三个列表，在它们之间插入+，将得到：

```
   0 1 2 3 + 4 5 6 7 + 8 9 10 11
```

```
12 15 18 21
```

将动词summover应用于表格，给出各列的总和。如果你想求这些行的和呢？

```
   sumover"1 a
6 22 38
```

以上是值得思考的。首先给你的新动词起个名字，这样更容易谈论。

```
   sumrows =. sumover"1

   sumrows a
6 22 38
```

在NuVoc中查找/，注意秩信息给出了派生动词的秩。 所以，summover有一元秩_ (unbounded)。动词summover应用于它的整个参数，因此在参数的项之间插入它的原始动词+。

动词sumrows具有一元秩1，并应用于其参数的1元格。它应用于参数的每个1元格，给出部分结果， 然后将这些部分结果组装到结果框架中。不是将整个参数输入动词sumrows，而是输入1元格， 因此sumrows在1元格的项之间插入+。表格参数的1元格是表格的行，所以+被插入到行的项之间。可视化来看：

```
sumrows   0 1 2 3   (first 1-cell)  gives  6

sumrows   4 5 6 7   (next 1-cell)   gives  22

sumrows   8 9 10 11 (next 1-cell)   gives  38
```

然后将部分结果6、22和38组装到结果列表中。

如果参数的秩是3呢？

```
   a =. i. 2 3 4
```

```
   a
 0  1  2  3
 4  5  6  7
 8  9 10 11

12 13 14 15
16 17 18 19
20 21 22 23
   sumover a
12 14 16 18
20 22 24 26
28 30 32 34
```

项是两个表格，在它们之间加个+就得到结果。因为这是所有项的和， 在这种情况下是对秩3阵列的表格的和，它可以被描述为对阵列的平面的和。

之所以使用sumover这个名字，是因为它在一开始就清除表达了所做的事情。 在实践中，直接使用原语可能更好。

```
   a =. i. 2 3 4
   +/ a NB. sum over planes
12 14 16 18
20 22 24 26
28 30 32 34
```

```
   +/"2 a  NB. sum over tables
12 15 18 21
48 51 54 57
   +/"1 a  NB. sum over rows
 6 22 38
54 70 86
```

第 79 章

表格化副词

双子v/为动词v计算一个表格（函数表）。

```
   a =. i. 5

   a +/ a   NB. addition table
0 1 2 3 4

1 2 3 4 5

2 3 4 5 6

3 4 5 6 7

4 5 6 7 8

   a */ a   NB. times table
0 0 0  0  0

0 1 2  3  4

0 2 4  6  8
```

```
0 3 6  9 12
0 4 8 12 16
  0 1 +/ 0 1    NB. + table on booleans
0 1
1 2
  0 1 +./ 0 1   NB. or
0 1
1 1
   0 1 +:/ 0 1  NB. not-or
1 0
0 0
   0 1 */ 0 1   NB. times
0 0
0 1
   0 1 *./ 0 1  NB. and
0 0
0 1
    0 1 *:/ 0 1 NB. not-and
1 1
```

```
1 0
```

派生动词v/左秩是v的左秩，右秩是_；它在左参数的每个单元格和整个右参数之间应用v。

```
   additiontable =. +/
```

动词additiontable被定义为副词/以+为参数应用后的结果。 该定义与你在前一节中使用的sumver的定义相同。addtiontable表示在使用双子，summover表示在使用单子。当然名字无法阻止对派生动词的误导使用。

```
   a additiontable a
0 1 2 3 4

1 2 3 4 5

2 3 4 5 6

3 4 5 6 7

4 5 6 7 8
```

+的左秩是0，所以派生动词的秩是0，它的单元格参数是原子。 派生动词的右秩是_，它的单元格参数是整个右参数。从视觉上看，上面的工作原理如下：

```
0 (first atom from left) + entire right   gives  0 1 2 3 4

1 (next atom from left)  + entire right   gives  1 2 3 4 5

2 (next atom from left)  + entire right   gives  2 3 4 5 6

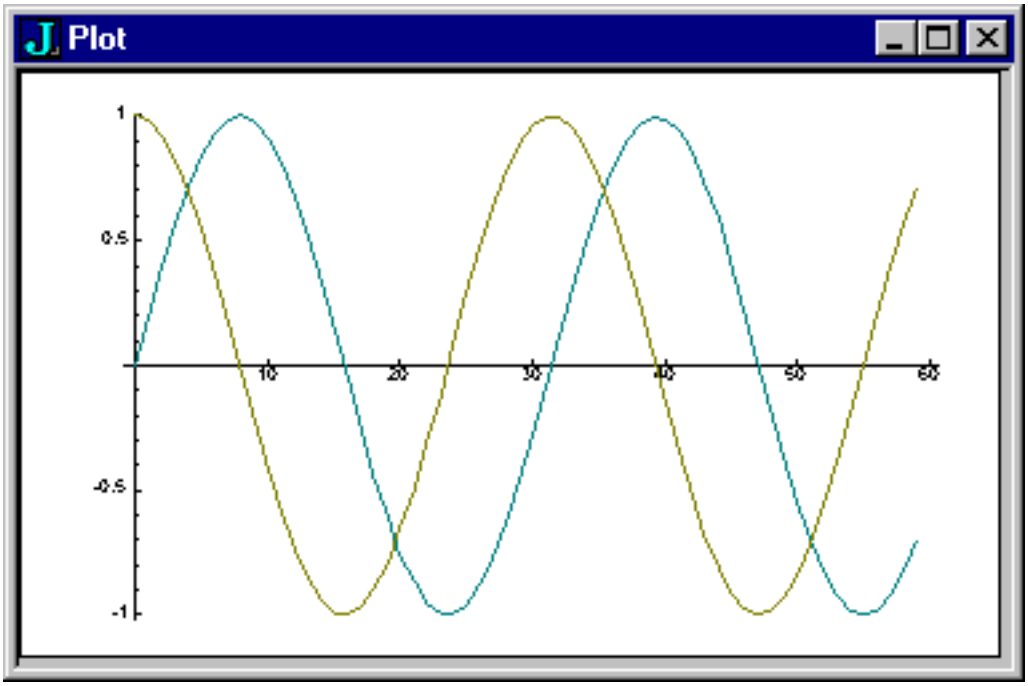
3 (next atom from left)  + entire right   gives  3 4 5 6 7

4 (next atom from left)  + entire right   gives  4 5 6 7 9
```

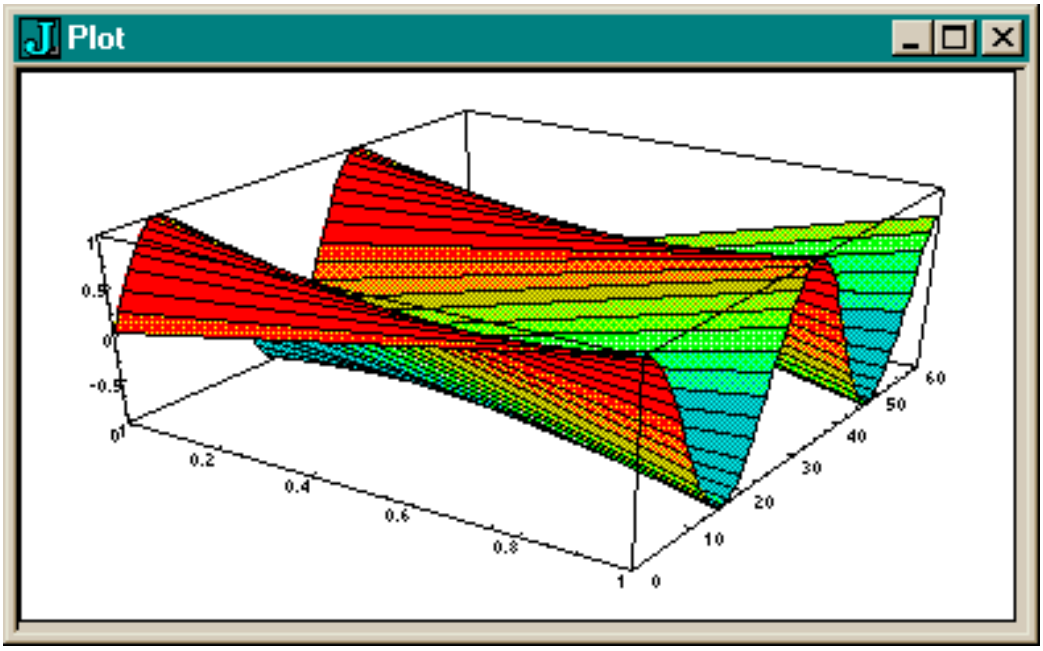
下面是一个有趣的表格化用法，用于前面章节的绘图原语。 绘图工具将表格参数中的每一行绘制为单独的数据系列。

```
load 'plot'

plot 1 2 o. / 0.2 * i.60
```




```
'surface' plot 1 2 o. / 0.2 * i.60
```



左参数中的1用作o.的左参数，给出第一行的正弦值。左边参数中的2用作o.的做参数，给出了第二行的余弦值。

在上面的例子中，原始动词的左秩总是0，因此派生动词的左参数的单元格就是左参数的原子。让我们看一个例子，其中原始动词的左秩不是0。

```
   f =. ,"1
```

动词f是左右秩分别为1的，(append)原语。

```
   p=. 2 2 $'abcd'

   p
ab
cd

   q=.3 3$'ABCD'

   q
```

```
ABC

DAB

CDA


   p f/ q

abABC

abDAB

abCDA


cdABC

cdDAB

cdCDA
```

可视化：

```
ab (1st list from left) , entire right   gives abABC
                                          abDAB
                                          abCDA

cd (next list)      , entire right   gives  cdABC
                                             cdDAB
                                             cdCDA
```

第 80 章

连词

连词类似于副词，不同之处在于它同时接受左、右两个参数。 在前面的章节中非正式地介绍了秩连词"。此外，用于定义动词（如摄氏度）的:现在可以被认为是连词。 在定义摄氏度时，:的左参数为3，右参数为0。因此看起来它可能是一个动词，但它的结果是一个动词的，这个事实证明它是一个连词。

连词的动词结果称为派生动词。

第 81 章

副词和连词的执行顺序

副词和连词的优先级高于动词。这意味着副词或连词在动词之前执行。 此外，副词或连词的左参数是它前面的整个动词短语。NuVoc的 Modifiers辅助页面给出了解析和执行的确切规则。

出于实践目的，下面用示例说明这些规则。

```
   a =. i.2 3

   +/"1 - a
```

```
_3 _12
```

与所有J句子一样，上面的句子从右到左执行。在将+解析为双子或单子之前，执行优先级更高的连词"。连词以1作为它的右参数， 以它左边的整个动词短语作为它的左参数。左边的动词短语是副词，它的左参数是+。下面使用括号来明确根据规则所确定的执行顺序。

```
   ((+/)"1) (- a)
```

如前所述，带有常量的简单例子可能需要将作为连词宾语的常量与作为派生动词宾语的常量分开。

```
   a (+"1) 1 2 3
```

```
a +"1 (1 2 3)

a +"1 [ 1 2 3
```

最后一个使用原语单子[(same)来返回它的参数。这样比较短，并且避免了括号的使用。

第 82 章

装盒 - 单子 <

到目前为止，你已经处理了数字或字符原子。单子< (box)引入了一种称为盒装的新原子类型。 单子<适用于任何名词并返回一个原子，该原子是一个包含参数的盒子。

阵列要么是包含数字的数值阵列，要么是包含字符的文字阵列，要么是包含盒子的盒装阵列。 数字和字符阵列被称为开着的，用以区分盒装阵列。

```
   b =. < 2 3 4

   $b       NB. an atom has empty shape
```

盒装阵列显示在盒子中。

```
   b
+-----+
|2 3 4|
+-----+

   c =. < 4 7 9

   d =. b , c   NB. append
```

```
   d
+-----+-----+
|2 3 4|4 7 9|
+-----+-----+
   $d       NB. list with shape 2
2
   (< 2 3 4) , < 4 7 9
+-----+-----+
|2 3 4|4 7 9|
+-----+-----+
```

不同类型（数字、字符和盒装）的阵列不能相互拼接。

```
   'a' , 3
|domain error, executing dyad ,
|arguments are incompatible: character and numeric
|   'a'    ,3
   3 , < 2 3 4
|domain error, executing dyad ,
|arguments are incompatible: character and boxed
|   'a'    ,<2 3 4
```

盒装阵列具有相同的类型，无论它们包含什么都可以拼接。

```
   (< 'abc') , < 4 5 6
+---+-----+
|abc|4 5 6|
+---+-----+
   (< 2 3 $ 'abcdef') , (< i. 3 4) , < 23
+---+---------+--+
|abc|0 1  2  3|23|
|def|4 5  6  7|  |
|   |8 9 10 11|  |
+---+---------+--+
```

第 83 章

链接 - 双子 ;

双子; (link)可以很容易地创建一个盒装名词列表。

```
   2 3 4 ; 4 7 9
+-----+-----+
|2 3 4|4 7 9|
+-----+-----+
   'abc' ; 5 7 9
+---+-----+
|abc|5 7 9|
+---+-----+
   (2 3 $ 'abcdef') ; (i. 3 4) ; 23
+---+---------+--+
|abc|0 1  2  3|23|
```

```
|def|4 5  6  7|  |
|   |8 9 10 11|  |
+---+---------+--+
```

在NuVoc中查找链接。 注意它的二元秩是_ _，所以它适用于整个左参数和整个右参数。它总是把左参数装盒，而右参数如果已经是盒装，则不会被装盒。 只有当右参数不是盒装时，才会对它进行装盒，从而使以下操作运行：

```
   'abc' ; 'defg' ; 'hijkl'
+---+----+-----+
|abc|defg|hijkl|
+---+----+-----+
```

从右到左计算为：

```
   'abc' ; ('defg' ; 'hijkl')
```

所以最左边;的右参数已经是盒装了，就不会再被装盒了。再次装盒的话，会是什么样子？

```
   'abc' ; < 'defg' ; 'hijkl'
+---+------------+
|abc|+----+-----+|
|   ||defg|hijkl||
|   |+----+-----+|
+---+------------+
```

盒子可以嵌套！

第 84 章

开盒 - 单子 >

单子> (open)是装盒的逆操作。也就是说，它把盒子里的东西取出来。把它用在已经开盒的名词上是没有效果的。

```
   > 23 5 7
23 5 7
   a =. i.5
   a
0 1 2 3 4
   < a
+---------+
|0 1 2 3 4|
+---------+
   > < a
0 1 2 3 4
```

在NuVoc中查找>。 单子>的秩为0，所以它应用于参数中的每个原子。每个原子被打开，创建一个部分结果，然后与所有其他部分结果组合成最终结果。

```
   a =. 1 2 3 ; 5 6 7

   a

+-----------+

|1 2 3|5 6 7|

+-----------+

   > a

1 2 3

5 6 7
```

如果部分结果都具有相同的秩和形，则直接组装它们。但是，开盒的部分结果可能会有非常不同的秩和形。

```
   a =. 1 ; 2 3 ; 4 5 6

   a

+-----------+

|1|2 3|4 5 6|

+-----------+

   > a

1 0 0
```

```
2 3 0

4 5 6
```

打开每个原子，并扩展部分结果以适应结果框架。

```
> 1     gives    1   (atom)

> 2 3   gives   2 3 (list)

> 4 5 6 gives   4 5 6   (list)
```

结果框架必须具有具有相同秩和形的部分结果。为此，将每个部分结果扩展为具有与最高秩结果的秩相同的秩， 并且每个部分结果的形在任何轴上都取最大值。

在上面的示例中，原子1被扩展为一个列表，并用0填充以获得形为3的列表。 用0填充列表2 3，使其形为3。列表4 5 6无需填充。最终结果的框架为3， 其中每个结果单元格是一个包含3个原子的列表。

如果盒子中包含字符呢？

```
   > 'abc' ; 'defg' ; 'hijkl'

abc

defg

hijkl
```

结果是一个3 × 5的表格。字符部分结果以类似的方式进行扩展，但使用空白作为填充，而不是0。 不能打开同时包含字符和数值数据的盒装列表。

```
   > 1 2 3 ; 'asdf'

|domain error, executing monad >

|contents are incompatible: numeric and character
```

```
|       >1 2 3;'asdf'
```

没有办法把数值和字符部分结果放在一起，所以这是一个域错误。

第 85 章

取自 - 二元 {

双子{ (from)用于从阵列中选择项。这有时被称为索引阵列。

```
   a =. 5 7 9 4 8

   3 { a    NB. select item at index 3
4

   0 { a        NB. indexes start at 0
5

   2 4 { a  NB. select 2 items
9 8

   2 0 4 { a    NB. select 3 items
9 5 8
```

{从其参数中选择它的项，因此从表格中选择的是行。

```
   m =. i. 3 4
```

```
   m
0 1  2  3
4 5  6  7
8 9 10 11
   0 { m
0 1 2 3
   2 1 { m
8 9 10 11
4 5  6  7
```

{从秩为3的阵列中选择的是表格。

```
   1 { i. 2 3 4
12 13 14 15
16 17 18 19
20 21 22 23
```

第 86 章

取自（盒装索引）

如果{的左参数是盒装的，则打开它，并且它的每个项沿着右参数的轴依次给出索引。 这可以用于从阵列中选择任何子阵列。

```
   m =. i. 3 4

   m

0 1  2  3

4 5  6  7

8 9 10 11

   (< 1 2) { m

6
```

打开的做参数是列表1 2。第一项为1，作为第一个轴的索引；第二个项是2，作为第二个轴的索引。 1选择列表4 5 6 7，2从列表中选择6。

如果列表中的项数少于轴数，则选择所有尾轴。

```
   (<1) { m

4 5 6 7
```

对于更高秩的阵列，会更有趣。

```
   d =. i.2 3 4

   d

 0  1  2  3

 4  5  6  7

 8  9 10 11


12 13 14 15

16 17 18 19

20 21 22 23

   (<1 2 3) { d NB. plane 1, row 2, column 3

23

   (<1 2) { d   NB. plane 1, row 2, all columns

20 21 22 23
```

到目前为止，索引列表中每个轴的项都是一个原子，并且只选择一个项。如果需要选择多个项怎么办？ 如果索引列表中的项被装盒，那么它就是该轴的索引列表。假设要从原子表格m中选择0 2行、0 2 3列。即表格：

```
      col-0 col-2  col-3

row-0   0     2      3

row-2   8    10     11
```

轴的索引为列表：

```
   0 2 ; 0 2 3

+---+-----+

|0 2|0 2 3|

+---+-----+
```

该索引列表需要装盒，以便将每个项视为连续轴的索引。0 2选择行（第一个轴），0 2 3选择列（第二个轴）。

```
   < 0 2 ; 0 2 3

+-----------+

|+---+-----+|

||0 2|0 2 3||

|+---+-----+|

+-----------+

   (< 0 2 ; 0 2 3) { m

0  2  3

8 10 11

   (< 0 1 ; 0 2 ; 2 3) { d

 2  3

10 11


14 15
```

```
22 23
```

同样，如果项数少于轴数，则选择所有尾轴。

```
   (< 0 1 ; 0 2) { d

 0  1  2  3

 8  9 10 11


12 13 14 15

20 21 22 23
```

通常期望的子阵列包括不是尾轴的轴的所有项。这可以通过给出该轴的所有索引来实现。

```
   (< 0 1 2 ; 2 3) { m

 2  3

 6  7

10 11
```

在需要计算索引的实际应用程序中，这可能不方便。因此，一个装盒的空列表<''表示选择了轴中的所有项。

```
      < (<'') ; 2 3

+--------+

|+--+---+|

||++|2 3||
```

```
|||||    ||

||++|    ||

|+--+---+|

+--------+
```

上面的参数用作左参数，将选择第一个轴的所有项，因为第一个索引是一个盒装空列表， 并沿第二个轴选择第2个项和第3个项。

盒装空列表非常有用，因此将原语a:定义为<''。因此，上面的写法可以简化。

```
  (< a: ; 2 3) { m

 2  3

 6  7

10 11
```

以上可以更容易地用连词" (rank)实现。

```
   2 3 {"1 m

 2  3

 6  7

10 11
```

第 87 章

取自（分散索引）

如果左参数是单体，则打开它，并且它的项是沿连续轴的索引。

```
      (<0 2 ; 2 3) { m

 2  3

10 11
```

如果左参数不是单体的呢？

```
   (0 2 ; 2 3) { m

2 11
```

这是怎么回事？没什么特别的，因为这只是你的老朋友" (rank)。 双子{的左秩是0，右秩是_。这意味着左参数被当作0元格，右参数被整体拿来使用。可视化看：

```
(<0 2)  (first left cell)  { m   gives 2

(<2 3)  (next left cell)   { m   gives 11
```

结果由部分结果2和11组合而成。

这被称为分散索引。

```
   (0 0 ; 1 1 ; 2 2) { m      NB. scatter index a diagonal

0 5 10
```

第 88 章

修订 }（修改选中的）

} (amend)是一个副词，其结果是一个双子，用于修改阵列。 amend的左参数通常是名词。让我们来看一个例子：

```
   change_index_two =. 2}
```

动词change_index_two用作双子。它的右参数是原数据，左参数是为索引位置2准备的新值。

```
   15 change_index_two 5 6 7 8
```

```
5 6 15 8
```

```
   30 change_index_two 23 18 17
```

```
23 18 30
```

```
   'b' change_index_two 'cat'
```

```
cab
```

```
   15 (2}) 5 6 7 8
```

```
5 6 15 8
```

```
   30 (2}) 23 18 17

23 18 30

   'b' 2} 'cat'

cab
```

它以你可能期望的方式扩展。

```
   23 (1 4}) 7 7 7 7 7

7 23 7 7 23

   23 24 (1 4}) 7 7 7 7 7

7 23 7 7 24

   'bet' 2 5 8} 'cattumbiz'

cabtuebit
```

一般来说，修正x s} y定义为：

> 通过将y中被s选择的部分替换为x来形成结果。}的s参数与动词{的左参数处理方式相同。

修订允许我们为阵列的选定部分赋予新值。修订的参数提供关于要修改的数据的索引信息。 这用于选择要修改的元素，就像将其用作二元动词{的左参数一样。

如果你理解{ (from)，那么修订就很简单了。

```
   m =. i. 3 4

   1 { m

4 5 6 7
```

```
   23 23 23 23 (1}) m

 0  1  2  3

23 23 23 23

 8  9 10 11

   23 (1}) m

 0  1  2  3

23 23 23 23

 8  9 10 11
```

你可以先使用选择信息来查看要修改的数据。

```
   1 2 { m

4 5  6  7

8 9 10 11
```

所选数据是形为2 4的子阵列。因此，你需要一个形为2 4的子阵列来替换所选数据。

```
  (2 4 $ 23 23 23 23 24 24 24 24) (1 2}) m

 0  1  2  3

23 23 23 23

24 24 24 24
```

假设要修改第1行和第2行以及第0列和第3列的子阵列，该子阵列的新值为12。

```
   12 ((<1 2;0 3)}) m

 0 1  2  3

12 5  6 12

12 9 10 12
```

修改该子阵列，将其替换为左参数中的阵列。

```
   (2 2 $ 23 24 25 26) ((<1 2;0 3)}) m

 0 1  2  3

23 5  6 24

25 9 10 26
```

第 89 章

取自 { 之外的选择方式

有些使用{ (from)的情况非常常见，所以它们有自己的原语。 这里没有对这些原语进行任何详细的讲解，提到这些原语是为了让你了解它们， 可以查找它们的定义并且在你自己的工作中使用它们。这里通过例子来说明。

单子{. (head)取其参数的第一个项，类似于{左参数为0。

```
   {. 5 6 7
5
   {. i. 3 4
0 1 2 3
```

单子{: (tail)取其参数的最后一项，类似于{左参数为_1（哦，对了，前面忘了提，负索引是从轴的末端开始索引）。

```
   {: 5 6 7
7
   {: i. 3 4
8 9 10 11
```

单子}. (behead)删除其参数的第一项。

```
   }. 5 6 7
6 7
   }. i. 3 4
4 5  6  7
8 9 10 11
```

单子}: (curtail)删除其参数的最后一项。

```
   }: 5 6 7
5 6
   }: i. 3 4
0 1 2 3
4 5 6 7
```

双子{. (take)从左参数所列的轴获取指定数量的项。

```
   3 {. i. 8
0 1 2
   2 3 {. i. 3 4
0 1 2
4 5 6
   2 {. i. 3 4
```

```
0 1 2 3

4 5 6 7
```

双子}. (drop)从左参数所列的轴上删除指定数量的项。

```
   1 }. 2 3 4

3 4

   1 2 }. i. 3 4

 6  7

10 11
```

一个有趣的思考{., {:, }.和}:的方式是：它们对数组的角落进行索引。

双子{.的一个和{没有直接关系的功能是：它可以创建一个比选定角落更大的阵列。 它通过适当地填充0、''或a:来实现这一点。

```
   5 {. 5 6 7

5 6 7 0 0

   4 5 {. i. 2 3

0 1 2 0 0

3 4 5 0 0

0 0 0 0 0

0 0 0 0 0

   5 {. 'abc'

abc
```

```
   <"0 [ 5 {. 'abc'    NB. make sure they are there
+-+-+-+-+-+
|a|b|c| | |
+-+-+-+-+-+
   7 {. (<"0) 5 {. 'abc'   NB. fill with a:
+-+-+-+-+-+++
|a|b|c| | |||
+-+-+-+-+-+++
```

双子# (copy)也与{ (from)有相当直接的关系。它的左参数是一个列表，表示从右参数中重复相应项的次数。

```
   3 2 1 2 3 # 'abcde'
aaabbcddeee
```

以上内容相当于以下内容：

```
   0 0 0 1 1 2 3 3 4 4 4 { 'abcde'
aaabbcddeee
```

第 90 章

分割 ;.

;. (cut)连词将动词应用于其参数的分割。在讨论;. (cut)时，我们将使用表达式f =: u ;. n来给各个元素命名。 ;. (cut)的左参数是动词u，它将应用于派生动词的右参数的分割。;. (cut)的右参数是名词n，表示分割的种类。

让我们考虑n为1、_1、2和_2，以及衍生动词作为单子使用的情况。

如果n是2或_2，右参数的项被划分为以参数中的最后一项为分隔符的参数。用于标记分割的项称为品（乐器术语）。 负n表示该品不包括在分割中。每个分割被传递给动词u，部分结果被组装成最终结果。

```
   cut2  =. < ;. 2
```

cut2的定义可以理解为：box cut last。

```
   cut2 'how now brown cow '
```

```
+----+----+------+----+
|how |now |brown |cow |
+----+----+------+----+
```

在本例中，右参数的项是字符。品是最后一个字符，即一个空格。品用于

将整个参数分解为一系列参数， 动词<用于这些参数。视觉上:

```
< 'how '  (first partition)  gives  first partial result

< 'now '  (next partition)   gives  next partial result

< 'brown '

< 'cow '
```

部分结果被组合成最终结果。

```
   <;._2 'how now brown cow '

+---+---+-----+---+

|how|now|brown|cow|

+---+---+-----+---+
```

cut2装盒的结果包括品，而<;._2装盒的结果不包括品。

下面的代码应用于同样的右参数的分割，但是应用双子# (tally)。这为我们提供了每个分割的计数。

```
   #;._2 'how now brown cow '

3 3 5 3
```

这同样适用于数值数据。在下面的代码中，_1是品。

```
   a =. 2 3 4 _1 5 3 2 _1 23 45 65 132 _1

   <;._2 a

+-----+-----+------------+

|2 3 4|5 3 2|23 45 65 132|
```

```
+-----+-----+------------+
```

```
   #;._2 a
```

```
3 3 4
```

```
   +/;._2 a
```

```
9 10 265
```

1或_1使用第一项作为品。如果为正，则在结果中包含品，如果为负则不包含品。

```
   <;._1 'madam i''m adam'
```

```
+---+---+----++
```

```
|ada| i'| ada||
```

```
+---+---+----++
```

有时，分割信息与数据是分开的。分割信息可以在派生动词的左参数中提供，而不是使用数据中的品。 分割信息是布尔数据，其中1表示分割的开始（1或_1）或结束（2或_2）。

```
   1 0 0 0 1 0 0 <;.1 'abcdefg'
```

```
+----+---+
```

```
|abcd|efg|
```

```
+----+---+
```

```
   d =.  'the test is the thing'
```

```
   'the' E. d
```

```
1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0
```

```
   ('the' E. d) <;.1 d
```

```
+------------+---------+
|the test is |the thing|
+------------+---------+
```

上面的示例使用了一个新的原语E.，如果想知道它的额外的信息，可以在NuVoc中查找它。 然而，无需关注细节，你应该知道发生了什么。

将字符列表分割成盒子非常有用，以至于有一个叫做cutopen的标准库实用程序，它可以处理许多常见情况。例如:

```
   cutopen 'testing testing 1 2 3'
```

```
+-------+-------+-+-+-+
|testing|testing|1|2|3|
+-------+-------+-+-+-+
```

第 91 章

每个 each

对于盒装数据，能够对每个盒装数据的内容执行一些操作通常是很有用的。 这非常有用，以至于标准配置文件定义了一个名为each的副词来完成这个任务。 each的定义涉及到的内容比逆目前所了解的要多一些，但是不要担心细节，只要使用它就可以了。

副词each以一个动词作为它的左参数。派生动词应用于其参数的盒装内容。

```
   a =. 10 12 13 ; 2 3 ; 4 5 6 8 3

   a

+--------+---+---------+

|10 12 13|2 3|4 5 6 8 3|

+--------+---+---------+

   +/ each a    NB. sum over each

+--+-+--+

|35|5|26|

+--+-+--+
```

```
   */ each a    NB. times over each
+----+-+----+
|1560|6|2880|
+----+-+----+
   |. each a    NB. reverse each
+--------+---+---------+
|13 12 10|3 2|3 8 6 5 4|
+--------+---+---------+
   >./ each a   NB. max over each
+--+-+-+
|13|3|8|
+--+-+-+
```

前面的例子都是使用衍生动词的单子形式。下面的用法实用派生动词的双子形式。

```
   23 , each a     NB. append each
+-----------+------+------------+
|23 10 12 13|23 2 3|23 4 5 6 8 3|
+-----------+------+------------+
   5 < each a
```

```
+-----+---+---------+
|1 1 1|0 0|0 0 1 1 0|
+-----+---+---------+
   1 |. each a
+--------+---+---------+
|12 13 10|3 2|5 6 8 3 4|
+--------+---+---------+
```

看明白上面的新动词|. (reverse or rotate)了吗？在NuVoc里查一下。

第 92 章

挂钩

由两个或三个单词组成的一个序列，具有特殊的含义。由两个动词组成的序列是一个挂钩，其计算方法如下：

```
  (f g) y    evaluates as    y f g y

x (f g) y   evaluates as    x f g y
```

假设你想缩放一个数字列表，使结果是每个数字除以列表中的最大值。列表上的最大值由将副词/应用于动词>.得到的派生的动词给出。

```
   a =. 3 5 8 2 7

   maxover =. >./

   maxover a
```

```
8
```

要把参数除以参数的最大值，你可以使用：

```
   a % maxover a
```

```
0.375 0.625 1 0.25 0.875
```

上面的代码可以写成一个挂钩。

```
   (% maxover) a
```

```
0.375 0.625 1 0.25 0.875
```

现在可以将缩放定义为% >. /组成的挂钩。

```
   scale =. % >./

   scale a
```

```
0.375 0.625 1 0.25 0.875
```

第 93 章

分叉

由三个动词组成的序列是一个分叉，其运行方式如下：

```
  (f g h) y  evaluates as    (f y) g (h y)

x (f g h) y evaluates as    (x f y) g (x h y)
```

分叉的一个简单示例是三个动词的序列+/ % #。副词/接受它的左参数+并返回一个动词， 因此这是一个由三个动词组成的序列。让我们从单子的角度来研究一下分叉的用法。

```
   (f  g h) y evaluates as    (f  y) g (h y)

   (+/ % #) y   evaluates as    (+/ y) % (# y)
```

这可以读作：**参数的总和除以参数的个数**。这就是**均值**的定义。

```
   (+/ % #) 5 9 12

8.66667

   (+/ % #) i.9

4
```

现在可以将mean定义为带有+/ % #的分叉。

```
   mean =. +/ % #

   mean i.9
4
```

第 94 章

隐式定义

动词摄氏度通过连词:被显式定义。术语**显式**表示定义中动词的参数通过其名称x和y显式地引用。

在隐式定义中，参数没有名字，也不显式地出现在定义中。参数由定义的语法要求隐式引用。你已经使用了几个隐式定义。

```
plus =. +

sumover =. +/

maxover =. >./

scale =. % >./

mean =. +/ % #
```

以上都是隐式定义。并且不要通过名称来引用参数。在某些情况下，隐式定义比同等的显式定义更简单、更明显。 在更复杂的情况下，你可能需要一些经验才能适应一个隐式定义。这部分是因为你可能有显式定义的经验， 而很少有隐式定义的经验。此外，隐式定义往往是定义的更简洁和数学表达形式。在等效的隐式定义变得清晰之前， 可能需要经历更详细的创建显式定义的步骤。

让我们回顾一下华氏度，看看它是如何被隐式定义的。打开`cf.ijs`脚本，

查看华氏度定义。

```
fahrenheit =: 3 : 0

t1 =. y * 9

t2 =. t1 % 5

t3 =. t2 + 32

)
```

你可以从清理显式定义开始。现在你对J比较熟悉了，你可以将这些计算合并成一行。

```
fa =: 3 : '(y * 9 % 5) + 32'
```

括号是必需的，因为其中的计算必须在加32之前完成。

让我们稍微调整一下定义，使构建隐式定义的步骤更清晰一些。

```
fb =: 3 : '32 + ((9%5) * y)'
```

上面的代码可以理解为：**将32加到参数的九分之五上**。

所以，你需要一个add32动词和一个ninefifthtimes动词。你可以用绑定连词&来隐式构建这些动词。 带有常量左参数的绑定连词返回一个派生单子，该单子是带有常量左参数并需要右参数的动词。

```
   add32 =: 32 & +
```

这将add32定义为将32与其参数相加的单子。

```
   add32 12
```

```
44
```

```
   ninefifthstimes =: (9%5) & *
```

这给出了一个单子，它把其参数乘以9%5。

```
   ninefifthstimes 20
```

```
36
```

结合这些，你会得到：

```
   add32 ninefifthstimes 100
```

```
212
```

连词@将两个动词组合成一个派生动词，该派生动词将右边的动词应用于其参数，然后将左边的动词应用于该结果。

```
  (u @ v) y   evaluates as    u v y
```

使用连词@将两个动词组合起来，形成最终的定义。

```
   fc =: add32 @ ninefifthstimes

   fc 100
```

```
212
```

```
   fc _40
```

```
_40
```

```
   fc 0
```

```
32
```

显示动词fc，并注意其定义依赖于其他两个定义。

```
   fc
```

```
+-----+-----------------+
```

```
|add32|@|ninefifthstimes|

+-----+-----------------+
```

有时，在你从较小的构建块构建了一个隐式定义之后，你会意识到真的不希望所有这些较小的定义到处存在。 副词f.以一个隐式定义为参数，并将其中的名字替换成对应的定义。

```
   fz =. fc f.
```

副词f.像所有副词一样，其参数在它的左边。

看一下fz的最终定义。

```
   fz

+--------+-+---------+

|+--+-+-+|@|+---+-+-+|

||32|&|+|| ||1.8|&|*||

|+--+-+-+| |+---+-+-+|

+--------+-+---------+
```

系统可以以几种不同的形式显示隐式定义。这些选项可以通过Edit | Session Display Form | Boxed来选择， 你将得到上面的显示。盒装在理解隐式定义方面非常有用。不过，现在选择Linear，这样你会看到以下内容：

```
   fz

32&+@(1.8&*)
```

在比较像定义为+的动词这样简单的东西时，隐式定义要比等效的显式定义简单得多。在华氏度的例子中， 可以认为显式定义更简单，特别是如果

你直接使用1.8而不是像默认定义那样使用9%5。

```
fx =: 3 : '32+1.8*y'
```

vs.

```
fz =: 32&+@(1.8&*)
```

隐式编程的真正优势在于对参数进行更复杂的转换，特别是当参数必须被多次引用时。下面举例说明了隐式定义的另一种用法。

```
   xmean =: 3 : '(+/y) % #y'
```

这是你们在分叉章节遇到的平均值。

```
   mean =: +/ % #
```

隐式定义只是直接使用分叉。

分叉也可以在显式定义中使用，但需要在分叉周围加上括号。

```
   xmean =: 3 : '(+/ % #)y'
```

隐式定义的一个优点是，与显式定义相比，它们更容易以正式的方式进行操作。例如， J可以自动派生许多隐式定义的逆定义。让我们用隐式定义fz尝试一下。华氏度的逆是摄氏度的数。 标准配置文件定义了副词inverse。

```
   fz =: 32&+@(1.8&*)

   cz =: fz inverse

   fz 100
```

```
212
```

```
   cz 212
```

```
100
```

```
fz 0
```

```
32
```

```
cz 32
```

```
0
```

隐式编程非常强大，但没有必要深入它。重要的知道它是什么，并在简单的情况下开始使用它， 因为这是熟悉它的最好方法。

第 95 章

显式-隐式转换器

有一个原语可以自动将单行显式定义转换为等效的隐式定义。通过编写一行显式定义，将它们转换为隐式形式， 并研究得到的隐式定义，逆可以学到很多关于隐式编程的知识。

我们用华氏度的显示定义来试一下。:的左参数3创建显式定义。:的左参数13创建隐式定义。

```
   fx =:  3 : '32 + y * 9 % 5' NB.  3 explicit

   ft =: 13 : '32 + y * 9 % 5'  NB. 13 tacit
```

使用Edit | Session Display Form | Linear显示。

```
   ft
32 + 1.8 * ]
```

乍一看，这是令人困惑的，因为它一次引入了几个新东西。首先要做的是查看盒装显示。

使用Edit | Session Display Form | Boxed选择盒装显示。

```
   ft
+--+-+---------+
```

```
|32|+|+---+-+-+|

|  | ||1.8|*|]||

|  | |+---+-+-+|

+--+-+---------+
```

在盒装的顶层有3个盒子。这是一个有三个要素的序列：noun verb verb。序列noun verb verb当作noun” _ verb verb， 这是一个分叉。你可以把它拆开，给各个部分命名，然后分开看。

分叉的第一个元素是短语32"_。给它起个名字，试着把它当作一个单子动词。

```
left =: 32"_

left 123
```

```
32
```

```
left i.5
```

```
32
```

不管你给left什么参数，它都会返回32。你以前见过连词"，但不是带常数左参数。 我们在NuVoc里查一下。当你翻到定义"的秩信息时， 你会注意到定义有三页，每页都是单独页面。这三页是：

```
Rank    u " n

Rank    m " n

Assign rank m " v  u " v  mv lv rv
```

不同的定义是用于不同类型参数的秩连词。在标题中，m和n表示名词参数，u和v表示动词参数。 你之前使用的"包含一个动词左参数和一个名词

右参数涵盖在第一个定义中。32和_ (infinity)都是名词， 所以和第二个定义相关。

阅读m"n的定义可以清楚地看出观察结果是正确的。派生动词的右秩为_，应用于它的整个右参数， 且无论右参数是什么，都会返回左参数，即32。

让我们看看分叉的右参数。

```
   right =: 1.8 * ]

   right 23
```

```
41.4
```

```
   right 10
```

```
18
```

让我们不要担心定义的细节，而是再一次通过观察，动词right所做的是将其参数乘以1.8（即9%5）。

最后一个定义是个分叉。

```
   ff =: left + right

   ff 100
```

```
212
```

```
   ff 0
```

```
32
```

比较上一课中的手动隐式定义与自动翻译的隐式定义，并注意它们的不同之处。

```
32&+@(1.8&*)

32+1.8*]
```

隐式编程丰富多样，与bond、atop、rank等副词和连词以及hook、fork等序列紧密相连。

第 96 章

检查点六

到了这个检查点，你应该明白：

- 术语副词和连词
- 如何求值带有副词和连词的句子
- 如何从阵列中选择子阵列
- 如何修改阵列中的子阵列
- 盒装数据
- 挂钩和分叉
- 隐式定义

通过做以下练习来检查你的理解：

- 实验双子`{ {. }.`（用负数做做参数）
- 在NuVoc查找`a.`，并用它作为双子`{ {. }`实验种的右参数
- 实验一下目前介绍的副词和连词
- 尝试使用挂钩`= <.`，确定它做什么以及为什么这样做（提示：给它一个名称并应用于`2.4 5.2 7 6.5`）
- 对挂钩`= +`做同样的事

第 97 章

外部 !:

J与环境交互是由连词!: (foreign)完成的。在大多数情况下，外部结果是提供特定服务的动词。 foreign的左参数选择一个通用的服务族，右参数选择一个特定的服务。由左参数选择的服务族是：

```
 0  scripts

 1  files

 2  host commands

 3  types

 4  names

 5  representations

 6  time

 7  space

 9  global parameters

11  windows
```

```
 13  debug

 15  dynamic link library (shared library)

 18  locales

128  numerical functions
```

NuVoc中记录了外部连词的概述， J词典的词汇表中有更详细的文档。

在前面的章节中，你已经遇到了一些外部连词的具体用法。

派生动词9!:11用于打印精度控制。9选择全局参数族，11给出一个派生动词，用于设置打印精度。

加载动词使用0!:0加载脚本。左参数选择脚本族。查看定义。 右参数0被视为三个十进制数字000，表示从文件中执行句子，在出现错误时停止，并且进行静默加载（不显示句子或结果）。

第 98 章

文件

许多应用程序需要读写文件。与所有和环境交互的J接口一样， 文件是通过外部连词访问的。外部服务族1操作文件。为了方便起见，首先定义几个动词。

```
readfile =: 1!:1

writefile =: 1!:2
```

让我们创建一个文件，其中包含一些数据。你将多次使用该文件名，因此请给它起一个名称。 文件外部操作要求文件名是一个盒装字符串。

```
fn =. < 'user\test.txt'

'testing 1 2 3' writefile fn
```

使用你喜欢的任何文件编辑器查看在J用户目录中创建的文件test.txt。你也可以使用File | open in Folder命令在J中将其作为脚本文件打开（你必须更改文件导航器上的首选项使其列出所有文件，以便查看文件test.txt文件）。

你可以从这个文件中读取数据。

```
data =. readfile fn
```

```
   data
testing 1 2 3
```

你可以重写文件并读取新数据。

```
   'new stuff for the file' writefile fn
   readfile fn
new stuff for the file
```

使用编辑器更改并重新保存test.txt文件，并再次读取该文件以查看是否获得了新数据。 同样，你可以通过在J中将该文件作为脚本文件打开、编辑、关闭并保存更改来完成此操作。

假设你有一个数字表，希望将其作为文本文件输出。

```
   numtab =. i. 4 5
   numtab writefile fn
|domain error
|   numtab     writefile fn
```

如果你尝试将numtab写出来，会得到一个域错误，因为writefile需要一个字符串作为其左参数。 因此，你需要将数字表转换为字符串。第一步是将数字数据转换为字符数据。原语": (format)负责这件事。

```
   cdata =. ": numtab
   cdata
 0  1  2  3  4
 5  6  7  8  9
```

```
10 11 12 13 14

15 16 17 18 19
```

```
   $cdata
```

```
4 14
```

cdata的显示看起来像数字数据，但它的形为4 14，这表明它是一个字符表格。 但是，你仍不能将其写入文件，因为写入文件必须是列表，而不是表格。

单子，(ravel)将阵列中的所有原子放入列表中。

```
   crdata =. , cdata
```

```
   $ crdata
```

```
56
```

```
   crdata writefile fn
```

```
   readfile fn
```

```
 0  1  2  3  4 5  6  7  8  910 11 12 13 1415 16 17 18 19
```

数据已写入文件。然而，从文件中读取数据显示仍然存在一些问题。 有四行数字的事实已经丢失，并且来自一行末尾的一些数字（例如9） 正好紧跟下一行的第一个数字。重要信息丢失。字符列表应该表明它有四行数据。

不同的平台使用不同的字符来标记行间的换行符。Unix系统使用新行字符（通常称为换行）。 Macintosh系统使用回车符。Windows系统使用回车和换行两个字符序列。标准配置文件将它们命名为LF、CR和CRLF。

J系统可以使用任何换行规范，但通常在内部使用LF，并仅在数据移入和移出J应用程序时在LF和主机偏好之间进行转换。

```
   'abc' , LF , 'defghi'
```

```
abc

defghi
```

想要对于cdata中的每个项（列表）附加LF，你需要使用秩为1版本的append来完成此操作。

```
   ddata =. cdata ,"1 LF

   ddata
 0  1  2  3  4

 5  6  7  8  9

10 11 12 13 14

15 16 17 18 19
```

显示中的空行出现是因为LF字符导致出现新行，而表的行尾也导致出现新行。 但是，当你利用它为创建列表时，系统将不会为此困扰，并且显示将再次看起来正常。

```
   ldata =. , ddata

   ldata
 0  1  2  3  4

 5  6  7  8  9

10 11 12 13 14

15 16 17 18 19
```

现在你有了一个包含原始数据完整信息的字符串。标准配置文件动词toHOST将 带有LF换行符的字符串转换为主机首选换行符。

```
(toHOST ldata) writefile fn
```

在编辑器中打开该文件，或者以脚本文件的形式打开该文件，查看数据是否存在。

如果你有了这个文件，并希望将其中的数字输入到J中进行处理，该怎么办？你需要逆转前面的过程。

```
   rdata =. readfile fn

   rdata

 0  1  2  3  4

 5  6  7  8  9

10 11 12 13 14

15 16 17 18 19
```

获取原始字符数据很容易。但是请注意，从形状来看，它是一个字符数据列表，并记住， 它的换行符可能有LF、CR或CRLF。标准配置文件动词toJ将各种换行符转换为LF。

```
   dlf =. toJ rdata

   dlf

 0  1  2  3  4

 5  6  7  8  9

10 11 12 13 14
```

```
15 16 17 18 19
```

可以使用cutopen动词对数据进行分割。

```
   bdata =. cutopen dlf

   bdata
+---------+---------+-------------+-------------+
|0 1 2 3 4|5 6 7 8 9|10 11 12 13 14|15 16 17 18 19|
+---------+---------+-------------+-------------+
```

每个盒子包含对应行的字符数据。你需要一个将字符串转换为数字的原语。双子".可用于将字符转换为数字。

```
   0 ". '5 2 7'
5 2 7

   a =. 0 ". '5 2 7'

   3 + a
8 5 10
```

".的左参数表示在数字转换失败时使用的值。

```
   0 ". '5 7.5 23.b 8'
5 7.5 0 8
```

使用each副词将每个盒子转换为数字。

```
   ndata =. 0 ". each bdata

   ndata
```

```
+---------+---------+-------------+-------------+
|0 1 2 3 4|5 6 7 8 9|10 11 12 13 14|15 16 17 18 19|
+---------+---------+-------------+-------------+
```

bdata和ndata的显示看起来是一样的，但是bdata盒子包含字符，ndata盒子包含数字。打开ndata盒子以获得数字表格。

```
   d =. > ndata

   d

 0  1  2  3  4

 5  6  7  8  9

10 11 12 13 14

15 16 17 18 19
```

你可以创建一个新的脚本文件，输入以下定义，并使用一个永久名称保存它，从而将所有这些内容包装在一起。

```
writetable =: dyad : 0

d =. ": x

d =. d ,"1 LF

d =. toHOST , d

d 1!:2 y

)

readtable =: 3 : 0
```

```
d =. 1!:1 y

d =. toJ d

d =. cutopen d

d =. 0 ". each d

d =. > d

)
```

运行脚本文件并测试你的定义。

```
   (i. 3 7) writetable fn

   1 + readtable fn

 1  2  3  4  5  6  7

 8  9 10 11 12 13 14

15 16 17 18 19 20 21
```

当你对J有了更多的经验，你就会开始把句子组合在一起。一个更有经验的J程序员可能会这样写上面的定义：

```
writetable =: 4 : '(toHOST,(": x),"1 LF) 1!:2 y'

readtable =: 3 : '>0 ". each cutopen toJ 1!:1 y'
```

脚本文件files.ijs为处理文件提供了许多有用的实用程序。 在J在线文档中查找它们。

```
   load 'files'

   (": i. 3 9) fwrites 'newtest.txt'
```

```
84

   0 ". freadr 'newtest.txt'

 0  1  2  3  4  5  6  7  8

 9 10 11 12 13 14 15 16 17

18 19 20 21 22 23 24 25 26
```